

Team Note of Junho0219

Kim Junho

Compiled on October 4, 2026

Contents

1 Data Structure	1	5.9 NTT	19
1.1 pbds	1	5.10 CRT	19
1.2 XOR Basis	2	5.11 Miller-Rabin	19
1.3 DSU Potential	2	5.12 Pollard's rho	20
1.4 Fenwick	2	5.13 Tonelli-Shanks	20
1.5 Segtree Walking	2	5.14 Cornacchia	20
1.6 Sweeping Segtree	2	5.15 Floorsum	20
1.7 PST	2	5.16 Xudyh's Sieve	21
1.8 treap	3	5.17 Lagrange Interpolation	21
1.9 Cartesian Tree	3	5.18 Moebius Inversion	21
1.10 Line Container	3	5.19 Math Theory	22
2 DP	4	6 String	22
2.1 knuth opt	4	6.1 Rolling Hash	22
2.2 DnC opt	4	6.2 KMP	23
2.3 hirschberg	4	6.3 Aho-Corasick	23
3 Geometry	4	6.4 Z	23
3.1 Header	4	6.5 Manacher	23
3.2 Smallest Enclosing Circle	5	6.6 Suffix Array	23
3.3 Segment Intersection	5	6.7 String Theory	24
3.4 Convex Hull	5	7 Misc	24
3.5 Rotating Calipers	5	7.1 Gray Code	24
3.6 DnC	6	7.2 Majority Vote	24
3.7 PIP	6	7.3 Ternary Search	24
3.8 PICP	6	7.4 RMQ	24
3.9 Tangent	6	7.5 Deque Trick	24
3.10 Bulldozer	7	7.6 Random	24
3.11 HPI	7	7.7 Time	24
3.12 3D Header	8	7.8 Debug	24
3.13 3D Convex Hull	9	1 Data Structure	
3.14 Green's Theorem	9	1.1 pbds	
3.15 Geometry Theory	10	<code>#include <ext/rope></code>	
4 Graph	10	<code>using namespace __gnu_cxx;</code>	
4.1 Bellman-Ford	10	<code>rope<char> rp;</code>	
4.2 SPFA	10	<code>rope<char> rp("string literal");</code>	
4.3 SCC	10	<code>rope<char> rp(st.c_str()); // rp(st)는 안 됨</code>	
4.4 2CC	10	<code>rp.push_back(ch);</code>	
4.5 BCC	10	<code>rp.insert(idx, string or rope or char);</code>	
4.6 Articulation Points	11	<code>rp.erase(pos, cnt); // pos부터 cnt개 삭제</code>	
4.7 Bridges	11	<code>rp.substr(pos, cnt);</code>	
4.8 2-SAT	11	<code>rp.substr(pos); // rp[idx] 문자 하나. string에서</code>	
4.9 Euler Circuit	12	<code>st.substr(idx)가 st[idx:]을 의미했던 것과는 다름에 주의</code>	
4.10 C3, C4	12	<code>rope<char> rp = rp1 + rp2;</code>	
4.11 PEO	13	<code>cout << rp;</code>	
4.12 Bimatch	13	<code>for (auto it = rp.begin(); it != rp.end(); it++) //</code>	
4.13 Hopcroft Karp	13	<code>rp[idx]접근은 O(logN)이므로 순회 시 iterator사용 // it++는</code>	
4.14 Dinic	13	<code>Amortized O(1)</code>	
4.15 MCMF	14	<code>#include <ext/pb_ds/assoc_container.hpp></code>	
4.16 MCMF(dijkstra)	14	<code>#include <ext/pb_ds/tree_policy.hpp></code>	
4.17 LCA	15	<code>using namespace __gnu_pbds;</code>	
4.18 LCA(RMQ)	15	<code>template <typename T> using ordered_set = tree<T, null_type,</code>	
4.19 ETT	15	<code>less<T>, rb_tree_tag,</code>	
4.20 HLD	15	<code>tree_order_statistics_node_update>;</code>	
4.21 Centroid Decomposition	15	<code>rb_tree_tag, tree_order_statistics_node_update>;</code>	
4.22 Virtual Tree	16	<code>// find_by_order(k) : k(0-based)번째 값의 it 반환</code>	
4.23 Graph Theory	16	<code>// order_of_key(x) : x보다 작은 원소 개수 반환</code>	
5 Math	16	<code>ordered_set os;</code>	
5.1 Linear Sieve	16	<code>os.insert(key);</code>	
5.2 Multiplicative Function	16	<code>os.erase(os.find_by_order(k));</code>	
5.3 Burnside's Lemma	16	<code>os.erase(os.find_by_order(os.order_of_key(key))); // key 삭제</code>	
5.4 Freivalds' Algorithm	16	<code>1</code>	
5.5 egcd	17	<code>int order = os.order_of_key(key); // key 삭제 2</code>	
5.6 FFT	17	<code>(들어있는값인지 확인후삭제)</code>	
5.7 FFT Division	17	<code>if (order < os.size()) {</code>	
5.8 Multipoint Evaluation	19	<code>auto it = os.find_by_order(os.order_of_key(key));</code>	
		<code>if (*it == key) os.erase();</code>	
		<code>}</code>	
		<code>int order = os.order_of_key(key); // 검색</code>	

```
if (order < os.size() && *os.find_by_order(order) == val) ;
```

1.2 XOR Basis

```
vector<T> basis;
auto insert = [&](T x) {
    for (auto e : basis) x = min(x, x ^ e);
    if (x) basis.push_back(x);
};
T mxSum = 0;
auto b = basis; // insert를 계속해야하는 경우 basis의 순서는 바뀌면 안됨
sort(b.rbegin(), b.rend());
for (auto e : b) mxSum = max(mxSum, mxSum ^ e);
```

1.3 DSU Potential

```
template <typename T>
struct DSU {
    vector<int> p;
    vector<T> w;
    DSU(int n) : p(n + 1), w(n + 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int a) {
        if (p[a] == a) return a;
        int ra = find(p[a]); // w[p[a]] 계산
        w[a] += w[p[a]];
        return p[a] = ra;
    }
    void merge(int a, int b, T val) { // b = a + val
        int ra = find(a);
        int rb = find(b);
        w[rb] = w[a] + val - w[b];
        p[rb] = ra;
    }
    T getDiff(int a, int b) {
        int ra = find(a);
        int rb = find(b);
        assert(ra == rb);
        return w[b] - w[a];
    }
};
```

1.4 Fenwick

```
template <typename T, int D>
struct FenwickTree {
    vector<FenwickTree<T, D - 1>> tree;
    template <typename... Args>
    FenwickTree(int n, Args... args) : tree(n + 1,
        FenwickTree<T, D - 1>(args...)) {}
    template <typename... Args>
    void update(int pos, Args... args) {
        for (; pos < tree.size(); pos += (pos & -pos))
            tree[pos].update(args...);
    }
    template <typename... Args>
    T sum(int r, Args... args) const {
        T res = 0;
        for (; r; r -= (r & -r)) res +=
            tree[r].query(args...);
        return res;
    }
    template <typename... Args>
    T query(int l, int r, Args... args) const {
        return sum(r, args...) - sum(l - 1, args...);
    }
};

template <typename T>
struct FenwickTree<T, 0> {
    T val = 0;
    void update(T v) { val += v; }
    T query() const { return val; }
};

// ex) 3D fenwick
// FenwickTree<int, 3> fw(n, m, k);
// fw.update(x, y, z, add); // O(logN logM logK)
// fw.query(x1, x2, y1, y2, z1, z2); // O(logN logM logK)
```

1.5 Segtree Walking

```
template <typename F>
int findRightMost(int l, F f) const { // f([l:r])=true인 최대
r // 만족하는 r 없다면 l-1리턴
    int node = 1 | sz;
    T acc = e();
    node >>= __builtin_ctz(node);
    while (f(op(acc, tree[node]))) {
```

```
        acc = op(acc, tree[node++]);
        if (!(node & (node - 1))) return n - 1;
        node >>= __builtin_ctz(node);
    }
    while (node < sz) {
        node <<= 1;
        if (f(op(acc, tree[node]))) acc = op(acc,
            tree[node++]);
    }
    return (node ^ sz) - 1;
}

template <typename F>
int findLeftMost(int r, F f) const { // f([l:r])=true인 최소 l
// 만족하는 l 없다면 r+1리턴
    int node = r | sz;
    T acc = e();
    node = max(1, node >> __builtin_ctz(~node));
    while (f(op(tree[node], acc))) {
        if (!(node & (node - 1))) return 0;
        acc = op(tree[node--], acc);
        node = max(1, node >> __builtin_ctz(~node));
    }
    while (node < sz) {
        node = node << 1 | 1;
        if (f(op(tree[node], acc))) acc = op(tree[node--],
            acc);
    }
    return (node ^ sz) + 1;
}

int findKthSmallest(int l, T k) const { // query([l, r]) >=
k인 최소 r
    if (query(l, n - 1) < k) return -1;
    return findRightMost(l, [&](T sum) { return sum < k; }) +
        1;
}

int findKthLargest(int r, T k) const { // query([l, r]) >= k인
최대 l
    if (query(0, r) < k) return -1;
    return findLeftMost(r, [&](T sum) { return sum < k; }) -
        1;
}
```

1.6 Sweeping Segtree

```
int sz = 1;
while (sz < n) sz <<= 1;
vector<int> tree(sz << 1), cnt(sz << 1);
auto update = [&](auto &&update, int node, int s, int e, int
l, int r, int add) -> void {
    if (l > e || s > r) return;
    if (l <= s && e <= r) cnt[node] += add;
    else {
        int m = s + e >> 1;
        update(update, node << 1, s, m, l, r, add);
        update(update, node << 1 | 1, m + 1, e, l, r, add);
    }
    if (cnt[node]) tree[node] = e - s + 1;
    else tree[node] = s != e ? tree[node << 1] + tree[node <<
1 | 1] : 0;
};
```

```
auto query = [&]() { return tree[1]; };
```

1.7 PST

```
struct Node {
    int l, r;
    ll val;
    Node() { l = r = val = 0; }
};

vector<Node> tree;
auto newNode = [&]() {
    tree.emplace_back();
    return tree.size() - 1;
};

auto upd = [&](auto &&upd, int old, int node, int s, int e,
int i, int add) {
    if (s == e) {
        tree[node].val = tree[old].val + add;
        return;
    }
    int m = s + e >> 1;
    if (i <= m) {
        tree[node].r = tree[old].r;
        upd(upd, tree[old].l, tree[node].l = newNode(), s, m,
            i, add);
```

```

    }
    else {
        tree[node].l = tree[old].l;
        upd(upd, tree[old].r, tree[node].r = newNode(), m + 1,
            e, i, add);
    }
    tree[node].val = tree[tree[node].l].val +
        tree[tree[node].r].val;
};
auto qry = [&](auto &qry, int node, int s, int e, int l, int
r) {
    if (l <= s && e <= r) return tree[node].val;
    if (l > e || s > r) return 0LL;
    int m = s + e >> 1;
    return qry(qry, tree[node].l, s, m, l, r) + qry(qry,
        tree[node].r, m + 1, e, l, r);
};
vector<int> roots;
roots.push_back(newNode()); // version 1
1.8 treap
constexpr ll MOD = 998'244'353;
struct Node {
    int l = -1, r = -1;
    ll sum, lm = 1, la = 0, val;
    int pr, sz = 1;
    bool flip = false;
    inline static mt19937 rng;
    Node(int val) : sum(val), val(val), pr(rng()) {}
    void propagate();
    void recalc();
};
vector<Node> tree;
int cnt(int node) { return ~node ? tree[node].sz : 0; }
void Node::propagate() {
    if (flip) {
        swap(l, r);
        if (~l) tree[l].flip ^= 1;
        if (~r) tree[r].flip ^= 1;
        flip = 0;
    }
    if (lm != 1 || la) {
        val = (val * lm + la) % MOD;
        sum = (sum * lm + la * sz) % MOD;
        for (auto nxt : {l, r}) if (~nxt) {
            tree[nxt].lm = tree[nxt].lm * lm % MOD;
            tree[nxt].la = (tree[nxt].la * lm + la) % MOD;
        }
        lm = 1;
        la = 0;
    }
}
void Node::recalc() {
    sz = 1;
    sum = val;
    if (~l) {
        tree[l].propagate();
        sz += tree[l].sz;
        sum += tree[l].sum;
        if (sum >= MOD) sum -= MOD;
    }
    if (~r) {
        tree[r].propagate();
        sz += tree[r].sz;
        sum += tree[r].sum;
        if (sum >= MOD) sum -= MOD;
    }
}
pair<int, int> split(int node, int k) {
    if (!~node) return {-1, -1};
    tree[node].propagate();
    if (cnt(tree[node].l) >= k) {
        auto [a, b] = split(tree[node].l, k);
        tree[node].l = b;
        tree[node].recalc();
        return {a, node};
    }
    auto [a, b] = split(tree[node].r, k - cnt(tree[node].l) -
1);
    tree[node].r = a;
    tree[node].recalc();
    return {node, b};
}

```

```

int merge(int a, int b) {
    if (!~a) return b;
    if (!~b) return a;
    tree[a].propagate();
    tree[b].propagate();
    if (tree[a].pr > tree[b].pr) {
        tree[a].r = merge(tree[a].r, b);
        tree[a].recalc();
        return a;
    }
    tree[b].l = merge(a, tree[b].l);
    tree[b].recalc();
    return b;
}
void insert(int &root, int idx, int node) {
    auto [a, b] = split(root, idx);
    root = merge(merge(a, node), b);
}
void erase(int &root, int idx) {
    auto [a, bc] = split(root, idx);
    auto [b, c] = split(bc, 1);
    root = merge(a, c);
}
}

template <typename F>
void f(int &root, int l, int r, F f) {
    auto [a, bc] = split(root, l);
    auto [b, c] = split(bc, r - l + 1);
    assert(~b);
    f(b); // split후라서 b는 propagate된 상태
    root = merge(merge(a, b), c);
}
}

```

```

// reverse(root, l, r)
f(root, l, r, [&](int node) {
    tree[node].flip ^= 1;
});
// update(root, l, r, mul, add)
f(root, l, r, [&](int node) {
    tree[node].lm = mul;
    tree[node].la = add;
});
// query(root, l, r)
f(root, l, r, [&](int node) {
    cout << tree[node].sum << "\n";
});

```

1.9 Cartesian Tree

```

auto getCartesian(const vector<int> &v) { // min // 0-based
    int n = v.size();
    vector<pair<int, int>> chd(n, {-1, -1});
    vector<int> s;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && v[s.back()] > v[i]) {
            chd[i].first = s.back();
            s.pop_back();
        }
        if (!s.empty()) chd[s.back()].second = i;
        s.push_back(i);
    }
    return pair{s[0], chd}; // root, chd
}

```

// i의 왼쪽 직계 자식에서 오른쪽 방향으로만 이동하는 체인 위의 j에 대해 right_lt[j]=i
// i의 오른쪽 직계 자식에서 왼쪽 방향으로만 이동하는 체인 위의 j에 대해 left_le[j]=i

1.10 Line Container

// upper=false일 때 테스트안해봤음

```

struct Line { //
https://github.com/kth-competitive-programming/kactl/blob/main/content/mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
template <bool upper>
struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {

```

```

    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
}

void add(ll k, ll m) {
    if constexpr (!upper) k = -k, m = -m;
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
        isect(x, erase(y));
}

ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    if constexpr (upper) return l.k * x + l.m;
    else return -(l.k * x + l.m);
}
};

2 DP
2.1 knuth opt
const int INF = 2e9;
vector dp(k + 1, vector<int>(n + 1, INF));
for (int i = 0; i <= k; i++) dp[i][0] = 0;

auto getDP = [&](auto &&getDP, int i, int s, int e, int optL,
int optR) -> void {
    if (s > e) return;
    int m = s + e >> 1;
    int opt = -1;
    for (int j = max(lowerLimit(m), optL); j <=
min(upperLimit(m), optR); j++) {
        int val = dp[i - 1][j] + f(j + 1, m);
        if (dp[i][m] > val) {
            dp[i][m] = val;
            opt = j;
        }
    }
    assert(~opt);
    getDP(getDP, i, s, m - 1, optL, opt);
    getDP(getDP, i, m + 1, e, opt, optR);
};

for (int i = 1; i <= k; i++) getDP(getDP, i, 1, n, 0, n - 1);
// O(KN logN)
cout << dp[k][n];

// dp[i][k] = min_{low(k)<=j<=up(k)}{ dp[i - 1][j] + C(j, k) }
// dp[i][k]에 대해 dp[i - 1][j] + C(j, k)가 최소가 되는 가장
작은 j를 opt(i, k)라 할 때 opt(i, k) <= opt(i, k + 1)이어야 함
// 사각부등식(a<=b<=c<=d일 때 f(a,c)+f(b,d)<=f(a,d)+f(b,c))
만족하면 opt(i, j) <= opt(i, j + 1)

2.2 DnC opt
const int INF = 2e9;
vector dp(k + 1, vector<int>(n + 1, INF));
for (int i = 0; i <= k; i++) dp[i][0] = 0;

auto getDP = [&](auto &&getDP, int i, int s, int e, int optL,
int optR) -> void {
    if (s > e) return;
    int m = s + e >> 1;
    int opt = -1;
    for (int j = max(lowerLimit(m), optL); j <=
min(upperLimit(m), optR); j++) {
        int val = dp[i - 1][j] + f(j + 1, m);
        if (dp[i][m] > val) {
            dp[i][m] = val;
            opt = j;
        }
    }
    assert(~opt);
    getDP(getDP, i, s, m - 1, optL, opt);
    getDP(getDP, i, m + 1, e, opt, optR);
};

for (int i = 1; i <= k; i++) getDP(getDP, i, 1, n, 0, n - 1);
// O(KN logN)
cout << dp[k][n];

```

```

// dp[i][k] = min_{low(k)<=j<=up(k)}{ dp[i - 1][j] + C(j, k) }
// dp[i][k]에 대해 dp[i - 1][j] + C(j, k)가 최소가 되는 가장
작은 j를 opt(i, k)라 할 때 opt(i, k) <= opt(i, k + 1)이어야 함
// 사각부등식(a<=b<=c<=d일 때 f(a,c)+f(b,d)<=f(a,d)+f(b,c))
만족하면 opt(i, j) <= opt(i, j + 1)

```

2.3 hirschberg

```

vector<int> getDP(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<int> dp(m + 1);
    for (int i = 1; i <= n; i++) {
        vector<int> ndp(m + 1);
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) ndp[j] = dp[j - 1] + 1;
            else ndp[j] = max(ndp[j - 1], dp[j]);
        }
        swap(dp, ndp);
    }
    return dp;
}

string LCS(const string &a, const string &b) { // time O(NM),
space O(M)
    if (a.empty()) return "";
    if (a.size() == 1) return b.find(a) != string::npos ? a :
    "";
    int n = a.size(), m = b.size();
    int mid = n / 2;
    string l = a.substr(0, mid), r = a.substr(mid);
    auto dp1 = getDP(l, b);
    string rrev = r, brev = b;
    reverse(rrev.begin(), rrev.end());
    reverse(brev.begin(), brev.end());
    auto dp2 = getDP(rrev, brev);
    int mx = -1, mxIdx;
    for (int i = 0; i <= m; i++) {
        if (mx < dp1[i] + dp2[m - i]) {
            mx = dp1[i] + dp2[m - i];
            mxIdx = i;
        }
    }
    return LCS(l, b.substr(0, mxIdx)) + LCS(r,
b.substr(mxIdx));
}

3 Geometry
3.1 Header
template <typename T>
struct Point {
    T x, y;
    Point() = default;
    Point(T x, T y) : x(x), y(y) {}
    template <typename U> Point(const Point<U> &other) :
x(static_cast<T>(other.x)), y(static_cast<T>(other.y)) {}
    bool operator<(const Point &other) const { return tie(x,
y) < tie(other.x, other.y); }
    bool operator<=(const Point &other) const { return tie(x,
y) <= tie(other.x, other.y); }
    bool operator==(const Point &other) const { return tie(x,
y) == tie(other.x, other.y); }
    Point operator-(const Point &other) const { return {x -
other.x, y - other.y}; }
};

template <typename T>
T crossProduct(const Point<T> &p1, const Point<T> &p2) {
    return (p1.x * p2.y - p2.x * p1.y);
}

template <typename T>
int ccw(const Point<T> &p1, const Point<T> &p2, const Point<T>
&p3) { // -1 : 시계, 0 : 일직선, 1 : 반시계
    T cp = crossProduct(p2 - p1, p3 - p1);
    return (cp > 0) - (cp < 0);
}

template <typename T>
inline T distSquarePP(const Point<T> &p1, const Point<T> &p2)
{
    return (p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) *
(p2.y - p1.y);
}

template <typename T>
inline ld distPP(const Point<T> &p1, const Point<T> &p2) {

```

```

    return hypot<ld>(p2.x - p1.x, p2.y - p1.y);
}
template <typename T>
ld distPL(const Point<T> &p, const Point<T> &l1, const
Point<T> &l2) { // distance from P(point, p) to L(line, l1l2)
    assert(!(l1 == l2));
    return abs(crossProduct(l1 - p, l2 - p)) / distPP(l1, l2);
}
template <typename T>
T getPolygonAreaDouble(const vector<Point<T>> &polygon) {
    if (polygon.size() <= 2) return 0;
    T res = 0;
    for (int i = 0, j = polygon.size() - 1; i <
        polygon.size(); j = i++) res += crossProduct(polygon[j],
        polygon[i]);
    return abs(res);
}
ld heron(ld a, ld b, ld c) { // 헤론
    ld s = ld(0.5) * (a + b + c);
    return sqrt(s * (s - a) * (s - b) * (s - c));
}
ld brahmagupta(ld a, ld b, ld c, ld d) { // 브라마굽타,
cyclic이어야 함
    ld s = ld(0.5) * (a + b + c + d);
    return sqrt((s - a) * (s - b) * (s - c) * (s - d));
}
ld getSegmentCircleArea(ld r, ld len) { // 활꼴 넓이, r :
반지름, len : 활꼴 길이
    ld cosTheta = 1 - ld(len * len) / (2 * r * r);
    ld sinTheta = sqrt(1 - cosTheta * cosTheta); // > 0
    ld theta = acos(cosTheta);
    return ld(0.5) * r * r * (theta - sinTheta);
}
template <typename T>
bool isBetween(Point<T> a, Point<T> b, Point<T> c) {
    return min(a.x, c.x) <= b.x && b.x <= max(a.x, c.x) &&
        min(a.y, c.y) <= b.y && b.y <= max(a.y, c.y);
}
template <typename T>
bool isOnPL(Point<T> p, Point<T> l1, Point<T> l2) { // p가 l1
12위에 있는지
    return ccw(p, l1, l2) == 0 && isBetween(l1, p, l2);
}
ld getOtherSideLength(ld a, ld b, ld cosTheta) { // cos II
    return sqrt(a * a + b * b - 2 * a * b * cosTheta);
}
ld getCosTheta(ld x, ld y, ld z) { // x 맞은편 각도
    return (y * y + z * z - x * x) / 2 / y / z;
}
pll tangentMerge(l1 a, l1 b, l1 c, l1 d) { // tanT1 = a / b,
tanT2 = c / d, tan(T1 + T2) = ?
    return {
        (a * d % mod + b * c % mod) % mod,
        (b * d % mod - a * c % mod + mod) % mod
    }; // {분자, 분모}
}
Point<ld> footPL(const Point<ld> &p, const Point<ld> &p1,
const Point<ld> &p2) { // p에서 직선 p1p2에 내린 수선의발
    ld a = p2.y - p1.y;
    ld b = p1.x - p2.x;
    ld c = a * p.y - b * p.x;
    ld d = a * p1.x + b * p1.y;
    ld hx = (-b * c + a * d) / (a * a + b * b);
    ld hy = (a * c + b * d) / (a * a + b * b);
    return {hx, hy};
}
Point<ld> reflectPL(const Point<ld> &p, const Point<ld> &p1,
const Point<ld> &p2) { // p의 직선 p1p2 기준 대칭이동
    auto [hx, hy] = footPL(p, p1, p2);
    return Point<ld>{2 * hx - p.x, 2 * hy - p.y};
}
void rotate2D(ld &x, ld &y, ld theta) { // 반시계방향으로
theta만큼 회전
    tie(x, y) = pair<ld, ld>{cos(theta) * x + sin(theta) * y,
        -sin(theta) * x + cos(theta) * y};
}

```

3.2 Smallest Enclosing Circle

// 헤더: Point, Cross Product, CCW, distPP

```

template <typename T>
Point<ld> circumcircle(const Point<T> &p1, const Point<T> &p2,
const Point<T> &p3) {

```

```

    Point<T> b = p2 - p1, c = p3 - p1;
    T dist12 = b.x * b.x + b.y * b.y;
    T dist23 = c.x * c.x + c.y * c.y;
    T d = 2 * crossProduct(b, c);
    return Point<ld>(p1.x + ld(dist12 * c.y - dist23 * b.y) /
        d, p1.y + ld(dist23 * b.x - dist12 * c.x) / d);
}

```

```

template <typename T>
pair<Point<ld>, ld>
getSmallestEnclosingCircle(vector<Point<T>> points) { // 평균
0(N)
    mt19937
    rng(chrono::system_clock::now().time_since_epoch().count());
    shuffle(points.begin(), points.end(), rng);
    Point<ld> res;
    ld r = -1;
    for (int i = 0; i < points.size(); i++) if (distPP(res,
        points[i]) > r) {
        res = points[i], r = 0;
        for (int j = 0; j < i; j++) if (distPP(res, points[j])
            > r) {
            res = Point<ld>(ld(points[i].x + points[j].x) / 2,
                ld(points[i].y + points[j].y) / 2), r =
                distPP(res, points[i]);
            for (int k = 0; k < j; k++) if (distPP(res,
                points[k]) > r) res = circumcircle(points[i],
                points[j], points[k]), r = distPP(res, points[k]);
        }
    }
    return {res, r};
}

```

3.3 Segment Intersection

```

template <typename T>
bool checkIntersect(const Point<T> &p1, const Point<T> &p2,
const Point<T> &p3, const Point<T> &p4) {
    int a = ccw(p1, p2, p3) * ccw(p1, p2, p4);
    int b = ccw(p3, p4, p1) * ccw(p3, p4, p2);

    if (a > 0 || b > 0) return false;
    if (a < 0 || b < 0) return true;
    return min(p1, p2) <= max(p3, p4) && min(p3, p4) <=
        max(p1, p2);
}

```

3.4 Convex Hull

```

template <typename T>
vector<Point<T>> getConvexHull(vector<Point<T>> ps) { //
0(NlogN)
    sort(ps.begin(), ps.end());
    ps.erase(unique(ps.begin(), ps.end(), ps.end()));
    if (ps.size() <= 1) return ps;
    vector<Point<T>> h;
    for (auto x : ps) {
        while (h.size() >= 2 && ccw(h[h.size() - 2], h.back(),
            x) <= 0) h.pop_back();
        h.push_back(x);
    }
    int lower = h.size();
    for (int i = ps.size() - 2; i >= 0; i--) {
        while (h.size() > lower && ccw(h[h.size() - 2],
            h.back(), ps[i]) <= 0) h.pop_back();
        h.push_back(ps[i]);
    }
    h.pop_back();
    return h;
}

```

3.5 Rotating Calipers

// 헤더: Point, crossProduct, ccw
// getConvexHull

```

template <typename T>
T distSquare(const Point<T> &p1, const Point<T> &p2) {
    return (p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) *
        (p2.y - p1.y);
}
template <typename T>
T getDiameterSquare(const vector<Point<T>> &points, bool
isConvex=false) { // 0(NlogN)
// isConvex=true할 경우 points가 반시계 정렬되어있어야함.
시계정렬이면 그냥 false로 보내서 사용하기
    if (points.size() <= 1) return 0;

```

```

    if (points.size() == 2) return distSquare(points[0],
    points[1]);
    const vector<Point<T>> &v = isConvex ? points :
    getConvexHull(points);
    T diameter = 0;
    int n = v.size(), a = 0, b = 1;
    while (a < n && b < 2 * n) {
        diameter = max(diameter, distSquare(v[a % n], v[b %
        n]));
        if (crossProduct(v[(a + 1) % n] - v[a % n], v[(b + 1)
        % n] - v[b % n]) >= 0) ++b;
        else ++a;
    }
    return diameter;
}

template <typename T>
inline T getTriangleAreaDouble(const Point<T> &a, const
Point<T> &b, const Point<T> &c) {
    return abs(a.x * b.y + b.x * c.y + c.x * a.y - a.y * b.x -
    b.y * c.x - c.y * a.x);
}

template <typename T>
T largestTriangleDouble(const vector<Point<T>> &hull) { //
O(N^2) // 볼록껍질에 포함되는 가장 큰 삼각형의 넓이
    assert(hull.size() >= 3);
    T res = 0;
    for (int i = 0; i + 2 < hull.size(); i++) {
        for (int j = i + 1, k = i + 2; j < hull.size(); j++) {
            while (k + 1 < hull.size() &&
            getTriangleAreaDouble(hull[i], hull[j], hull[k]) <
            getTriangleAreaDouble(hull[i], hull[j], hull[k +
            1])) ++k;
            res = max(res, getTriangleAreaDouble(hull[i],
            hull[j], hull[k]));
        }
    }
    return res;
}

```

3.6 DnC

```

constexpr ld eps = 1e-5;
ld dnc(vector<pair<ld, ld>> v) { // O(N log^2N)
    assert(v.size() > 1);
    sort(v.begin(), v.end(), [&](auto &a, auto &b) { return
    a.first < b.first; });
    auto f = [&](auto &&f, int s, int e) -> ld {
        if (s + 1 == e) return dist(v[s], v[e]);
        if (s + 2 == e) return min({
            dist(v[s], v[s + 1]),
            dist(v[s + 1], v[e]),
            dist(v[e], v[s])
        });
        int m = s + e >> 1;
        ld res = min(f(f, s, m), f(f, m + 1, e));
        vector<pair<ld, ld>> mids;
        for (int i = s; i <= e; i++) {
            if (abs(v[m].first - v[i].first) < res)
                mids.push_back(v[i]);
        }
        // TLE나면 merge사용
        sort(mids.begin(), mids.end(), [&](auto &a, auto &b) {
            return a.second < b.second; });
        for (int i = 0; i < mids.size() - 1; i++) {
            for (int j = i + 1; j < mids.size(); j++) {
                if (mids[j].second - mids[i].second >= res)
                    break;
                res = min(res, dist(mids[i], mids[j]));
            }
        }
        return res;
    };
    return f(f, 0, v.size() - 1);
}

```

3.7 PIP

```

// 헤더: Point, crossProduct, ccw
template <typename T>
int PIP(const Point<T> &point, const vector<Point<T>>
&polygon) { // -1 : 내부, 0 : 경계, 1 : 외부
    int n = polygon.size();
    assert(n >= 3);
    bool inside = false;

```

```

    auto isBetween = [](T a, T b, T c) { return min(a, c) <= b
    && b <= max(a, c); };
    for (int i = 0, j = n - 1; i < n; j = i++) {
        const auto &p1 = polygon[i];
        const auto &p2 = polygon[j];
        // 다각형 경계
        if (ccw(p1, point, p2) == 0 && isBetween(p1.x,
        point.x, p2.x) && isBetween(p1.y, point.y, p2.y))
            return 0;
        // 다각형 내부
        if (min(p1.y, p2.y) <= point.y && point.y < max(p1.y,
        p2.y)) {
            if (p1.y < p2.y) inside ^= ccw(p1, p2, point) < 0;
            else inside ^= ccw(p2, p1, point) < 0;
        }
    }
    return 1 - 2 * inside;
}

```

3.8 PICP

```

// 헤더: Point, crossProduct, ccw, isBetween, isOnPL
template <typename T>
int PICP(const Point<T> &point, const vector<Point<T>>
&polygon, int dir=0) { // -1 : 내부, 0 : 경계, 1 : 외부 //
O(logN)
    int n = polygon.size();
    assert(n >= 3);
    if (!dir) { // dir은 polygon에서 점들이 주어진 방향(반시계
    : 1, 시계 : -1, 모르는 경우(default) : 0)
        int i = 2;
        while (!dir && i < n) dir = ccw(polygon[0],
        polygon[1], polygon[i++]);
    }
    assert(dir != 0); // dir = 0이면 모든 점이 일직선 위에
    존재
    if (ccw(polygon[0], polygon[1], point) * dir < 0) return
    1;
    if (ccw(polygon[0], polygon[n - 1], point) * dir > 0)
        return 1;
    if (isOnPL(point, polygon[0], polygon[1])) return 0;
    if (isOnPL(point, polygon[0], polygon[n - 1])) return 0;
    int lo = 1, hi = n;
    while (lo + 1 < hi) {
        int mid = lo + hi >> 1;
        if (ccw(polygon[0], polygon[mid], point) * dir >= 0)
            lo = mid;
        else hi = mid;
    }
    if (hi == n) return 1; // polygon[0], polygon[n - 1],
    point가 일직선인 경우
    return -ccw(polygon[lo], polygon[hi], point) * dir;
}

```

// polygon에 collinear vertex 없어야됨. 있을때 테스트안해봄

3.9 Tangent

```

// 헤더: Point, crossProduct, ccw, isBetween
template <typename T>
pair<int, int> polygonTangent(const Point<T> &point, const
vector<Point<T>> &polygon) { // O(logN)
    // polygon이 반시계 방향 정렬되어있어야함
    int n = polygon.size();
    assert(n >= 3);
    auto _polygonTangent = [&](int up) {
        auto isTangent = [&point, &up](const Point<T> &a,
        const Point<T> &b, const Point<T> &c) {
            return up * (ccw(point, b, a)) >= 0 && up *
            (ccw(point, b, c)) >= 0;
        };
        int lo = 0, hi = n;
        if (isTangent(polygon.back(), polygon[0], polygon[1]))
            return 0;
        while (lo + 1 < hi) {
            int mid = lo + hi >> 1;
            if (isTangent(polygon[mid - 1], polygon[mid],
            polygon[(mid + 1) % n])) return mid;
            if (up * ccw(point, polygon[lo], polygon[lo + 1])
            < 0) { // up
                if (up * ccw(point, polygon[mid], polygon[(mid
                + 1) % n]) > 0) hi = mid;
                else if (up * ccw(point, polygon[mid],
                polygon[lo]) > 0) lo = mid;
                else hi = mid;
            }
        }
    };
}

```

```

    else { // down
        if (up * ccw(point, polygon[mid], polygon[(mid
+ 1) % n]) < 0) lo = mid;
        else if (up * ccw(point, polygon[mid],
polygon[lo]) < 0) lo = mid;
        else hi = mid;
    }
}
if (lo && isTangent(polygon[lo - 1], polygon[lo],
polygon[lo + 1])) return lo;
if (hi < n && isTangent(polygon[hi - 1], polygon[hi],
polygon[(hi + 1) % n])) return hi;
return -1;
};
int a = _polygonTangent(1);
if (!a) return {-1, -1}; // 다각형 내부
vector<int> candi;
for (int i = 1; i < n && ccw(point, polygon[a], polygon[(a
- i + n) % n]) == 0; i++) candi.push_back((a - i + n) %
n);
for (int i = 1; i < n && ccw(point, polygon[a], polygon[(a
+ i) % n]) == 0; i++) candi.push_back((a + i) % n);
for (auto e : candi) if (isBetween(point, polygon[e],
polygon[a])) a = e;
int b = _polygonTangent(-1);
assert (~b);
candi.clear();
for (int i = 1; i < n && ccw(point, polygon[b], polygon[(b
- i + n) % n]) == 0; i++) candi.push_back((b - i + n) %
n);
for (int i = 1; i < n && ccw(point, polygon[b], polygon[(b
+ i) % n]) == 0; i++) candi.push_back((b + i) % n);
for (auto e : candi) if (isBetween(point, polygon[e],
polygon[b])) b = e;
return {a, b};
}

// PICP 대응으로 쓸수있음
// auto [a, b] = polygonTangent(point, polygon);
// if (a == -1) // 내부
// else if (ccw(polygon[a], polygon[b], point) == 0) // 경계
// else // 외부

// collinear points 많으면 O(N)으로 늘어날 수 있으니 제거하고
사용

```

3.10 Bulldozer

```

// 헤더: Point
template <typename T>
struct SlopeWithIdx {
    int i, j; // idx
    T dx, dy; // 무조건 dx >= 0 되게 저장
    SlopeWithIdx(int i, int j, Point<T> dir) : i(i), j(j),
dx(dir.x), dy(dir.y) {}
    bool operator<(const SlopeWithIdx &other) const { return
tuple<T, int, int>{dy * other.dx, i, j} < tuple<T, int,
int>{other.dy * dx, other.i, other.j}; }
    bool operator==(const SlopeWithIdx &other) const { return
dy * other.dx == other.dy * dx; }
};

template <typename T>
void bulldozer(vector<Point<T>> &points) { // O(N^2 logN)
    int n = points.size();
    vector<int> pos(n);
    iota(pos.begin(), pos.end(), 0);
    sort(points.begin(), points.end());
    vector<SlopeWithIdx<T>> slopes;
    slopes.reserve(n * (n - 1) / 2);
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n;
j++) slopes.emplace_back(i, j, points[j] - points[i]);
    sort(slopes.begin(), slopes.end());
    for (int i = 0, j; i < slopes.size(); i = j) {
        for (j = i; j < slopes.size() && slopes[i] ==
slopes[j]; j++) {
            int &idx1 = pos[slopes[j].i], &idx2 =
pos[slopes[j].j];
            // 인접한 원소끼리 순서를 바꿔며 모든 정렬을
            순회하게 됨, 즉 abs(idx1-idx2)==1임
            swap(idx1, idx2);
            swap(points[idx1], points[idx2]);
        }
    }
}

```

```

// idx1이랑 idx2는 pos[]의 alias이므로 함부로 값을
수정하면 안됨
// 예를 들어 if (idx1 > idx2) swap(idx1, idx2)
같은 거 하면 안됨
// 그냥 안전하게 i1, i2 선언해서 사용하는 거 추천
int i1 = min(idx1, idx2);
int i2 = max(idx1, idx2);

```

```

}
// slopes[i]의 기울기를 기준으로 정렬되어있음
// 코드
}

// // 점들 뒤집는 nC2개의 모든 과정을 확인
// for (int i = 0; i < slopes.size(); i++) { // 이 경우
for (auto &slope : slopes)로 해도 무방
//     int &idx1 = pos[slopes[i].i], &idx2 =
pos[slopes[i].j];
//     swap(idx1, idx2);
//     swap(points[idx1], points[idx2]);
//     // 코드
// }
}

```

3.11 HPI

```

// 헤더: Point, crossProduct, ccw
const ld eps = 1e-9; // epsilon

template <bool isCCW>
struct HalfPlane {
    Point<ld> start, dir; // isCCW=true일 때 반평면은 dir을
반시계 90도 회전한 방향(dir 왼쪽)에 위치
    template <typename T>
    HalfPlane(const Point<T> &s, const Point<T> &e) :
start(s), dir(e - s) {
        ld len = hypot(dir.x, dir.y);
        dir.x /= len;
        dir.y /= len;
    }
    template <typename T>
    HalfPlane(T a, T b, T c) { // ax + by + c >= 0
        assert(a != 0 || b != 0);
        start = (a != 0 ? Point<ld>(-ld(c) / a, 0) :
Point<ld>(0, -ld(c) / b));
        // ax + by + c = 0이면 a(x+a) + b(y+b) + c =
ax+by+c+a^2+b^2>=0 이므로 (a, b)쪽이 반평면 방향이
되어야 함
        if constexpr (isCCW) dir = {b, -a};
        else dir = {-b, a};
        ld len = hypot(dir.x, dir.y);
        dir.x /= len;
        dir.y /= len;
    }
    template <typename T>
    static HalfPlane<isCCW> getBisector(const Point<T> &a,
const Point<T> &b) { // 선분 ab의 수직이등분선이면서 dir이
a를 가리키는 반평면 생성자
        ld sx = ld(a.x + b.x) / 2;
        ld sy = ld(a.y + b.y) / 2;
        if constexpr (isCCW) return
HalfPlane<isCCW>(Point<ld>(sx, sy), Point<ld>(sx +
(a.y - b.y), sy + (b.x - a.x)));
        else return HalfPlane<isCCW>(Point<ld>(sx, sy),
Point<ld>(sx - (a.y - b.y), sy - (b.x - a.x)));
    }
    void move(ld d) {
        if constexpr (isCCW) { // dir 반시계 90도 법선 벡터 :
(-dir.y, dir.x)
            start.x -= d * dir.y;
            start.y += d * dir.x;
        }
        else { // dir 시계 90도 법선 벡터 : (dir.y, -dir.x)
            start.x += d * dir.y;
            start.y -= d * dir.x;
        }
    }
    friend bool isParallel(const HalfPlane &a, const HalfPlane
&b) { return abs(crossProduct(a.dir, b.dir)) < eps; }
    friend bool isOpposite(const HalfPlane &a, const HalfPlane
&b) { return (a.dir.x * b.dir.x + a.dir.y * b.dir.y) <
-eps; }
    Point<ld> getPoint(ld k) const {
        return {start.x + k * dir.x, start.y + k * dir.y};
    }
}

```

```

}
friend Point<ld> intersectionHH(const HalfPlane &a, const
HalfPlane &b) { // assert (!isParallel(a, b))일 때만 호출
됨
    ld k = crossProduct(b.start - a.start, a.dir) /
crossProduct(a.dir, b.dir);
    return b.getPoint(k);
}
bool include(const Point<ld> &point) { // point가 반평면에
포함되는지
    if constexpr (isCCW) return crossProduct(dir, point -
start) > eps;
    else return crossProduct(dir, point - start) < -eps;
}
bool include(const HalfPlane &a, const HalfPlane &b) { //
a, b의 교점이 반평면에 포함되는지
    return include(intersectionHH(a, b));
}
};
template <bool isCCW>
vector<Point<ld>> HPI(vector<HalfPlane<isCCW>> planes, ld d=0,
bool sortedByAngle=false) { // 반평면들을 각각 거리 d만큼
움직였을 때 교집합 // O(N logN)
    if (!sortedByAngle) { // 이미 선분들이 정렬되어있다면(ex)
블록다각형의 변들을 순서대로 넣은 경우) sortedByAngle=true
사용
        auto checkQuadrant = [](const HalfPlane<isCCW> &hp) ->
bool {
            return hp.dir.y < 0 || (hp.dir.y == 0 && hp.dir.x
< 0); // PI <= atan2(dir) < 2 * PI
        };
        sort(planes.begin(), planes.end(), [&](const
HalfPlane<isCCW> &a, const HalfPlane<isCCW> &b) {
            bool aq = checkQuadrant(a), bq = checkQuadrant(b);
            if constexpr (isCCW) return aq != bq ? aq < bq :
crossProduct(a.dir, b.dir) > 0;
            else return aq != bq ? aq < bq :
crossProduct(a.dir, b.dir) < 0;
        });
    }
    vector<int> idxes; idxes.reserve(planes.size());
    for (int i = 0, j; i < planes.size(); i = j) {
        int cur = i;
        j = i + 1;
        while (j < planes.size() && isParallel(planes[i],
planes[j])) {
            assert(!isOpposite(planes[i], planes[j]) &&
"교집합이 다각형으로 제한되지 않고 무한히 넓음");
            if (planes[cur].include(planes[j].start)) cur = j;
            ++j;
        }
        idxes.push_back(cur);
    }
    assert(idxes.size() >= 3 && "교집합이 다각형으로 제한되지
않고 무한히 넓음");
    // assert(!isParallel(planes[0], planes.back()) &&
"교집합이 다각형으로 제한되지 않고 무한히 넓음");
    assert(!isParallel(planes[idxes[0]], planes[idxes.back()])
&& "교집합이 다각형으로 제한되지 않고 무한히 넓음");
    if (d) for (auto &plane : planes) plane.move(d);
    deque<HalfPlane<isCCW>> dq;
    for (auto idx : idxes) {
        auto &plane = planes[idx];
        while (dq.size() >= 2 && !plane.include(dq[dq.size() -
2], dq[dq.size() - 1])) dq.pop_back();
        while (dq.size() >= 2 && !plane.include(dq[0], dq[1]))
dq.pop_front();
        if (dq.size() < 2 || dq[0].include(dq.back(), plane))
dq.push_back(plane);
    }
    vector<Point<ld>> res;
    if (dq.size() < 3) return res; // 교집합의 크기가 점
하나에 해당하는 경우 실수오차로 인해 교집합이 없는 것으로
계산되는 경우 생김. 이 경우 distance=1e-6로 살짝 옮겨주면
잘 계산됨
    for (int i = 0, j = dq.size() - 1; i < dq.size(); j = i++)
    {
        assert(abs(crossProduct(dq[i].dir, dq[j].dir)) > eps);
        // TODO 이 줄 그냥 지울까?
        res.push_back(intersectionHH(dq[i], dq[j]));
    }
}

```

```

assert(dq[i].include(res[0]) && "교집합이 다각형으로
제한되지 않고 무한히 넓음");
return res; // 리턴값은 블록 검질 형태임, isCCW에 따라
블록검질 정렬방향 다름(isCCW=true면 반시계방향정렬,
isCCW=false면 시계방향정렬)
}

template <typename point_t, typename border_t>
vector<vector<Point<ld>>> getVoronoiDiagram(const
vector<Point<point_t>> &points, const vector<Point<border_t>>
&border, int dir=0) { // border은 시계/반시계 중 한 방향으로
정렬되어있어야 함 // O(N^2 logN) // 중복점있으면안됨
    int n = border.size();
    if (!dir) { // dir은 border에서 점들이 주어진 방향(반시계
: 1, 시계 : -1, 모르는 경우(default) : 0)
        int i = 2;
        while (!dir && i < n) dir = ccw(border[0], border[1],
border[i++]);
    }
    assert(dir != 0); // dir = 0이면 모든 점이 일직선 위에
존재
    n = points.size();
    vector<vector<Point<ld>>> res(n);
    for (int i = 0; i < n; i++) {
        auto [x1, y1] = points[i];
        vector<HalfPlane<true>> planes;
        if (dir == 1) for (int i = 0, j = border.size() - 1; i
< border.size(); j = i++)
            planes.emplace_back(border[j], border[i]);
        else for (int i = 0, j = border.size() - 1; i <
border.size(); j = i++) planes.emplace_back(border[i],
border[j]);
        for (int j = 0; j < n; j++) if (i != j)
            planes.emplace_back(HalfPlane<true>::getBisector(points[i],
points[j]));
        res[i] = HPI(planes);
    }
    return res;
}

/*
hpi()는 반평면 교집합에 해당하는 다각형의 각 꼭짓점들의 좌표를
한 방향으로 정렬해서 리턴
hpi()에서 반평면 교집합의 크기가 점 하나인 경우 교집합이
없다고 판정될 수 있음.
hpi(planes, 1e-6)처럼 각 변들을 미세하게 음수방향으로 이동시킨
뒤 계산하면 됨
HalfPlane(T a, T b, T c) 생성자 사용시 isCCW는 딱히 신경 쓸
필요없음
HalfPlane(Point<T> &s, Point<T> &e) 생성자 사용할 때는 isCCW
방향 정확히 명시해줘야 함
*/

vector<Point<T>> points(n);
for (auto &[x, y] : points) cin >> x >> y;
// 블록다각형의 꼭짓점들이 반시계방향 정렬된 경우
vector<HalfPlane<true>> planes;
for (int i = 0, j = n - 1; i < n; j = i++)
    planes.emplace_back(points[j], points[i]);
// 블록다각형의 꼭짓점들이 시계방향 정렬된 경우
vector<HalfPlane<false>> planes;
for (int i = 0, j = n - 1; i < n; j = i++)
    planes.emplace_back(points[j], points[i]);

3.12 3D Header

template <typename T>
struct Point3D {
    T x, y, z;

    Point3D() = default;
    Point3D(T x, T y, T z) : x(x), y(y), z(z) {}
    template <typename U> Point3D(const Point3D<U> &other) :
x(static_cast<T>(other.x)), y(static_cast<T>(other.y)),
z(static_cast<T>(other.z)) {}

    bool operator<(const Point3D &other) const { return tie(x,
y, z) < tie(other.x, other.y, other.z); }
    Point3D operator-(const Point3D &other) const { return {x
- other.x, y - other.y, z - other.z}; }
};

template <typename T>

```

```

Point3D<T> crossProduct(const Point3D<T> &p1, const Point3D<T>
&p2) {
    return {p1.y * p2.z - p1.z * p2.y, p1.z * p2.x - p1.x *
p2.z, p1.x * p2.y - p1.y * p2.x};
}
template <typename T>
inline T dotProduct(const Point3D<T> &p1, const Point3D<T>
&p2) {
    return p1.x * p2.x + p1.y * p2.y + p1.z * p2.z;
}
template <typename T>
ld getTriangleArea(const Point3D<T> &p1, const Point3D<T> &p2,
const Point3D<T> &p3) {
    auto [nx, ny, nz] = crossProduct(p2 - p1, p3 - p1);
    return hypot<ld>(nx, ny, nz) / 2;
}
template <typename T>
tuple<T, T, T, T> getEquation(const Point3D<T> &p1, const
Point3D<T> &p2, const Point3D<T> &p3) { // ax + by + cz + d =
0
    auto [a, b, c] = crossProduct(p2 - p1, p3 - p1);
    assert(a != 0 || b != 0 || c != 0); // a=b=c=0이면
collinear
    auto d = -(a * p1.x + b * p1.y + c * p1.z);
    return {a, b, c, d};
}
template <typename T1, typename T2>
ld distPF(const Point3D<T1> &p1, const Point3D<T2> &f1, const
Point3D<T2> &f2, const Point3D<T2> &f3) { // distance from
P(point) to F(face)
    auto normal = crossProduct(f2 - f1, f3 - f1);
    return abs<ld>(dotProduct(normal, p1 - f1)) /
hypot<ld>(normal.x, normal.y, normal.z);
}

```

3.13 3D Convex Hull

```

// naive O(N^4)
// 헤더: Point3D, crossProduct, dotProduct
template <typename T>
bool isExtreme(const vector<Point3D<T>> &points, const
Point3D<T> &p1, const Point3D<T> &p2, const Point3D<T> &p3) {
    auto normal = crossProduct(p2 - p1, p3 - p1);
    if (normal.x == 0 && normal.y == 0 && normal.z == 0)
return false; // collinear
    int cntPos = 0, cntNeg = 0;
    for (auto point : points) {
        cntPos += (dotProduct(normal, point - p1) > 0);
        cntNeg += (dotProduct(normal, point - p1) < 0);
    }
    return cntPos == 0 || cntNeg == 0;
}
template <typename T>
vector<tuple<int, int, int>> getConvexHullFaceIdxes(const
vector<Point3D<T>> &points) {
    vector<tuple<int, int, int>> res;
    int n = points.size();
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            for (int k = j + 1; k < n; ++k) {
                if (isExtreme(points, points[i], points[j],
points[k])) res.emplace_back(i, j, k);
            }
        }
    }
    return res;
}

// incremental O(N^2)$
template <typename T>
bool collinear(const Point3D<T> &p1, const Point3D<T> &p2,
const Point3D<T> &p3) {
    auto [x, y, z] = crossProduct(p2 - p1, p3 - p1);
    return abs(x) <= 1e-9 && abs(y) <= 1e-9 && abs(z) <= 1e-9;
}
template <typename T>
vector<tuple<int, int, int>>
getConvexHullFaceIdxes(vector<Point3D<T>> &points) {
    int n = points.size();
    assert(n >= 3);
    sort(points.begin(), points.end());
    vector<tuple<int, int, int>> res;
    vector<vector<bool>> edgeVisible(n, vector<bool>(n,
true));
}

```

```

auto add = [&points, &res, &edgeVisible](int a, int b, int
c) {
    res.emplace_back(a, b, c);
    edgeVisible[a][b] = edgeVisible[b][c] =
edgeVisible[c][a] = false;
};
for (int i = 2; i < n; i++) if (!collinear(points[0],
points[i], points[i+1])) {
    swap(points[2], points[i]);
    break;
}
assert(!collinear(points[0], points[1], points[2])) &&
"모든 점들이 한 직선 위에 있음";
add(0, 1, 2);
add(0, 2, 1);
for(int i = 3; i < n; i++) {
    vector<tuple<int, int, int>> invisibleFaces;
    for(auto [idx1, idx2, idx3] : res) {
        Point3D<T> normal = crossProduct(points[idx2] -
points[idx1], points[idx3] - points[idx1]);
        if(dotProduct(normal, points[i] - points[idx1]) >
1e-9) edgeVisible[idx1][idx2] =
edgeVisible[idx2][idx3] = edgeVisible[idx3][idx1]
= true;
        else invisibleFaces.emplace_back(idx1, idx2,
idx3);
    }
    res.clear();
    for(auto [idx1, idx2, idx3] : invisibleFaces) {
        if(edgeVisible[idx2][idx1]) add(idx2, idx1, i);
        if(edgeVisible[idx3][idx2]) add(idx3, idx2, i);
        if(edgeVisible[idx1][idx3]) add(idx1, idx3, i);
    }
    res.insert(res.end(), invisibleFaces.begin(),
invisibleFaces.end());
}
return res;
}

```

3.14 Green's Theorem

반시계방향의 조각적으로 매끄러운 단순폐곡선 C 와 C 로 유계된 영역 D 에 대해

$$\iint_D \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \right) dA = \int_C P dx + Q dy$$

다각형의 각 변에 대한 합으로 계산하면

$$\int_C P dx + Q dy = \sum_{i=1}^N \int_{(x_i, y_i)}^{(x_{i+1}, y_{i+1})} P dx + Q dy$$

Examples

1. Polar Moment

$$P = -\frac{1}{3}y^3, \quad Q = \frac{1}{3}x^3$$

$$\begin{aligned} \iint_D (x^2 + y^2) dA &= \oint_C \frac{1}{3}x^3 dy - \frac{1}{3}y^3 dx \\ &= \sum_{i=1}^N \int_{(x_i, y_i)}^{(x_{i+1}, y_{i+1})} \frac{1}{3}x^3 dy - \frac{1}{3}y^3 dx \end{aligned}$$

각 변에서

$$x = x_i + (x_{i+1} - x_i)t, \quad y = y_i + (y_{i+1} - y_i)t, \quad 0 \leq t \leq 1$$

$$dx = (x_{i+1} - x_i)dt, \quad dy = (y_{i+1} - y_i)dt$$

$$= \frac{1}{3} \sum_{i=1}^N \int_0^1 ((x_i + (x_{i+1} - x_i)t)^3 (y_{i+1} - y_i) - (y_i + (y_{i+1} - y_i)t)^3 (x_{i+1} - x_i)) dt$$

$$= \frac{1}{12} \sum_{i=1}^N (x_i y_{i+1} - x_{i+1} y_i) (x_i^2 + x_i x_{i+1} + x_{i+1}^2 + y_i^2 + y_i y_{i+1} + y_{i+1}^2)$$

2. First Moment

$$\iint_D x \, dA = \oint_C \frac{1}{2} x^2 \, dy = \frac{1}{6} \sum_{i=1}^N (y_{i+1} - y_i)(x_i^2 + x_i x_{i+1} + x_{i+1}^2)$$

Area

$$S = \iint_D 1 \, dA$$

$$P = -\frac{y}{2}, \quad Q = \frac{x}{2}, \quad \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \right) = 1$$

$$\therefore S = \int_C (P \, dx + Q \, dy) = \frac{1}{2} \oint_C (x \, dy - y \, dx)$$

Circular Arc Integral

$$x = x_c + r \cos \theta, \quad y = y_c + r \sin \theta$$

$$S = \frac{1}{2} \oint (x \, dy - y \, dx) = \frac{r}{2} [x_c \sin \theta - y_c \cos \theta + r \theta]_{\theta_1}^{\theta_2}$$

Line Segment Integral

$$v_x = x_2 - x_1, \quad v_y = y_2 - y_1$$

$$x = x_1 + v_x t, \quad y = y_1 + v_y t, \quad 0 \leq t \leq 1$$

$$S = \frac{1}{2} \oint (x \, dy - y \, dx) = \frac{1}{2} [(v_y x_1 - v_x y_1) t]_0^1$$

$$= \frac{1}{2} (y_2 x_1 - y_1 x_1 - x_2 y_1 + x_1 y_1) = \frac{1}{2} (x_1 y_2 - x_2 y_1)$$

3.15 Geometry Theory

(x, y) 의 (a, b) 기준 대칭이동: $((a^2 - b^2)(-x) - 2aby, (a^2 - b^2)(y) - 2abx)$

Barycentric Coordinates

$A(x_a, y_a), B(x_b, y_b), C(x_c, y_c)$ 인 $\triangle ABC$ 에서
 $\overline{BC} = a, \overline{CA} = b, \overline{AB} = c, \mathcal{A} = b^2 + c^2 - a^2, \mathcal{B} = c^2 + a^2 - b^2, \mathcal{C} = a^2 + b^2 - c^2$ 라 할 때 중심의 Barycentric 좌표 $(\alpha : \beta : \gamma)$

외심 $(\alpha : \beta : \gamma) = (a^2 \mathcal{A} : b^2 \mathcal{B} : c^2 \mathcal{C})$

내심 $(\alpha : \beta : \gamma) = (a : b : c)$

무게중심 $(\alpha : \beta : \gamma) = (1 : 1 : 1)$

수심 $(\alpha : \beta : \gamma) = (\overline{BC} : \overline{CA} : \overline{AB})$

(꼭짓점 A 맞은편의) 방심 $(\alpha : \beta : \gamma) = (-a : b : c)$

직교좌표계로 표현하면 $\left(\frac{\alpha x_a + \beta x_b + \gamma x_c}{\alpha + \beta + \gamma}, \frac{\alpha y_a + \beta y_b + \gamma y_c}{\alpha + \beta + \gamma} \right)$

3D Rotation

$$\begin{pmatrix} v_x^2(1-c) + c & v_x v_y(1-c) - v_z s & v_x v_z(1-c) + v_y s \\ v_x v_y(1-c) + v_z s & v_y^2(1-c) + c & v_y v_z(1-c) - v_x s \\ v_x v_z(1-c) - v_y s & v_y v_z(1-c) + v_x s & v_z^2(1-c) + c \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix}$$

4 Graph

4.1 Bellman-Ford

```
auto bellmanFord = [&](int s) { // O(VE)
    vector<dist_t> dist(n + 1, INF);
    dist[s] = 0;
    for (int _ = n - 1; _--;) {
        for (auto [u, v, w] : edges) if (dist[u] != INF)
            dist[v] = min(dist[v], dist[u] + w);
    }
    bool hasNegCycle = false;
    for (auto [u, v, w] : edges) hasNegCycle |= (dist[u] !=
        INF && dist[v] > dist[u] + w);
    return pair{hasNegCycle, dist};
};
```

4.2 SPFA

```
auto spfa = [&](int s) { // O(VE) // average O(V+E)
    vector<dist_t> dist(n + 1, INF);
    dist[s] = 0;
    queue<int> q;
    q.push(s);
    vector<int> cnt(n + 1);
    ++cnt[s];
```

```
vector<bool> inq(n + 1);
while (!q.empty()) {
    auto cur = q.front();
    q.pop();
    inq[cur] = false;
    for (auto [nxt, cost] : adj[cur]) if (dist[nxt] >
        dist[cur] + cost) {
        dist[nxt] = dist[cur] + cost;
        if (inq[nxt]) continue;
        if (++cnt[nxt] >= n) return pair{true, dist}; //
        hasNegCycle
        q.push(nxt);
        inq[nxt] = true;
    }
}
return pair{false, dist}; // {hasNegCycle, dist}
};
```

4.3 SCC

```
auto get_scc(int n, const vector<vector<int>> &adj) { // adj는
1-based // O(V+E)
    vector<int> dfsn(n + 1), sccn(n + 1, -1);
    vector<int> s(n);
    int top = 0, dfsi = 0, scci = 0;
    auto dfs = [&](auto &&dfs, int cur) -> int {
        int low = dfsn[cur] = ++dfsi;
        s[top++] = cur;
        for (auto nxt : adj[cur]) if (!sccn[nxt]) low =
            min(low, dfsn[nxt] ? dfsn[nxt] : dfs(dfs, nxt));
        if (low == dfsn[cur]) {
            do { sccn[s[--top]] = scci; } while (s[top] !=
                cur);
            ++scci;
        }
        return low;
    };
    for (int i = 1; i <= n; i++) if (!dfsn[i]) dfs(dfs, i);
    for (int i = 1; i <= n; i++) sccn[i] = scci - 1 - sccn[i];
    return pair{scci, sccn}; // 0-based // scci = scc 개수
};
// graphToDAG(n, adj, sccn)
// vector<vector<int>> sccs(scci);
// for (int i = 1; i <= n; i++)
// sccs[sccn[i]].push_back(i);
// vector<int> visited(sccs.size(), -1);
// vector<vector<int>> dag(sccs.size());
// for (auto &scc : sccs) for (auto u : scc) {
//     for (auto v : adj[u]) if (sccn[v] != sccn[u] &&
// visited[sccn[v]] != sccn[u]) {
//         visited[sccn[v]] = sccn[u];
//         dag[sccn[u]].push_back(sccn[v]);
//     }
// }
```

4.4 2CC

```
vector<int> dfn(n + 1), eccn(n + 1), s(n);
int dfsi = 0, ecci = 0, top = 0;
auto dfs = [&](auto &&dfs, int cur, int par) -> int {
    int low = dfn[cur] = ++dfsi;
    s[top++] = cur;
    for (auto next : adj[cur]) if (next != par) low = min(low,
        dfn[next] ? dfn[next] : dfs(dfs, next, cur));
    if (low == dfn[cur]) {
        ++ecci;
        do { eccn[s[--top]] = ecci; } while (s[top] != cur);
    }
    return low;
};
```

dfs(dfs, 1, -1);

4.5 BCC

```
auto get_BCC(int n, int m, const vector<vector<pair<int,
int>>> &adj) { // O(V + E)
    vector<vector<int>> bccs;
    vector<int> dfsn(n + 1, 0), s(m);
    int dfsi = 0, top = 0;
    auto dfs = [&](auto &&dfs, int cur, int pid) -> int {
        int low = dfsn[cur] = ++dfsi;
        for (auto [nxt, id] : adj[cur]) if (id != pid) {
            if (!dfsn[nxt]) {
                s[top++] = id;
                int nlow = dfs(dfs, nxt, id);
                low = min(low, nlow);
            }
        }
    };
    for (int i = 1; i <= m; i++) dfs(dfs, i, -1);
    return bccs;
};
```

```

        if (nlow >= dfsn[cur]) {
            vector<int> bcc;
            do { bcc.push_back(s[--top]); } while
            (s[top] != id);
            bccs.push_back(bcc);
        }
    }
    else {
        if (dfsn[nxt] < dfsn[cur]) s[top++] = id;
        low = min(low, dfsn[nxt]);
    }
}
return low;
};
for (int i = 1; i <= n; i++) if (!dfsn[i]) dfs(dfs, i, -1);
return bccs;
}
vector<int> getNodeOfBCC(int n, const vector<int> &bcc, const
vector<pair<int, int>> &edges) {
    static int t = 1;
    static vector<int> visited(n + 1);
    if (visited.size() < n + 1) visited.resize(n + 1); //
    테스트케이스 여러 개인 문제면 n 바뀔 수 있음
    vector<int> res;
    for (auto id : bcc) {
        auto [u, v] = edges[id];
        if (visited[u] != t) res.push_back(u);
        if (visited[v] != t) res.push_back(v);
        visited[u] = visited[v] = t;
    }
    ++t;
    return res;
    // int E = bcc.size(), V = getNodeOfBCC(n, bcc).size();라
    할 때
    // E = V : 모든 점점의 차수가 2인 사이클(simple graph면
    단순사이클)
    // E > V : 사이클을 가짐
    // E < V : 단절선(E=1, V=2) 뿐임
    // E=0, V=1 : 정점하나만 동떨어진 경우
}
// 2개 이상의 BCC에 포함되는 정점은 단절점이다.(역도 성립)
// $V=2$, $E=1$라면 해당 BCC는 단절선이다.(역도 성립)

// ex) BCC 사용해서 단절점, 단절선 개수 세기
int cuts = 0, bridges = 0;
auto bccs = get_BCC(n, adj);
vector<int> cnt(n + 1);
for (auto &bcc : bccs) {
    auto nodes = getNodeOfBCC(n, bcc, edges);
    int V = nodes.size();
    int E = bcc.size();
    for (auto e : nodes) cuts += (++cnt[e] == 2);
    bridges += (V == 2 && E == 1);
}
cout << cutVertexes << "\n"; // 단절점 개수
cout << bridges << "\n"; // 단절선 개수

```

4.6 Articulation Points

```

vector<int> getArticulationPoints(int n, const
vector<vector<int>> &adj) { // 0(V + E)
    vector<int> dfsn(n + 1);
    vector<bool> isCutVertex(n + 1);
    int dfsi = 0;
    function<int(int, int)> dfs = [&](int cur, int parent) ->
    int {
        int low = dfsn[cur] = ++dfsi;
        int child = 0;
        for (auto next : adj[cur]) if (next != parent) {
            if (!dfsn[next]) {
                ++child;
                int nextLow = dfs(next, cur);
                low = min(low, nextLow);
                isCutVertex[cur] = isCutVertex[cur] | (~parent
                && nextLow >= dfsn[cur]);
            }
            else low = min(low, dfsn[next]);
        }
        isCutVertex[cur] = isCutVertex[cur] | (!~parent &&
        child > 1);
        return low;
    };
}

```

```

for (int i = 1; i <= n; i++) if (!dfsn[i]) dfs(i, -1);
vector<int> res;
for (int i = 1; i <= n; i++) if (isCutVertex[i])
    res.push_back(i);
return res;
}
// 단절점의 조건은 `nextLow >= in[cur]` 단절선의 조건 `nextLow
> in[cur]`과 다름에 주의
// 단절선이기 위해선 간선 이전의 모든 정점에 갈 수 없어야
하므로 이전 정점인 cur도 갈 수 없어야 하고,
// 단절점의 경우 cur에 갈 수 있다고 해도 어차피 정점이
제거되므로 상관없음

```

4.7 Bridges

```

vector<pair<int, int>> getBridges(int n, const
vector<vector<int>> &adj) { // 0(V + E)
    vector<int> dfsn(n + 1);
    vector<pair<int, int>> res;
    int dfsi = 0;
    auto dfs = [&](auto &&dfs, int cur, int par) -> int {
        int low = dfsn[cur] = ++dfsi;
        for (auto nxt : adj[cur]) if (nxt != par) {
            if (!dfsn[nxt]) {
                int nextLow = dfs(dfs, nxt, cur);
                low = min(low, nextLow);
                if (nextLow > dfsn[cur]) res.emplace_back(cur,
                nxt);
            }
            else low = min(low, dfsn[nxt]);
        }
        return low;
    };
    for (int i = 1; i <= n; i++) if (!dfsn[i]) dfs(dfs, i, -1);
    for (auto &[a, b] : res) if (a > b) swap(a, b);
    return res;
}

```

4.8 2-SAT

```

struct TwoSat {
    int n;
    vector<vector<int>> adj;
    vector<int> scnn;
    TwoSat(int n) : n(n), adj(2 * n) {}
    int f(int a) {
        if (a > 0) return 2 * (a - 1);
        return 2 * (-a - 1) + 1;
    }
    // 전부 1-based
    void add2SAT(int a, int b) { // a or b가 true
        adj[f(-a)].push_back(f(b));
        adj[f(-b)].push_back(f(a));
    }
    void add1SAT(int a) { adj[f(-a)].push_back(f(a)); } // a가
    true
    void addEqual2SAT(int a, int b) { add2SAT(a, -b);
    add2SAT(-a, b); } // a == b 가 true
    void addNotEqual2SAT(int a, int b) { add2SAT(a, b);
    add2SAT(-a, -b); } // a != b 가 true // a, b 중 정확히
    하나만 true
    void add22SAT(int a1, int a2, int b1, int b2) {
    add2SAT(a1, b1); add2SAT(a1, b2); add2SAT(a2, b1);
    add2SAT(a2, b2); } // (a1 and a2) or (b1 and b2)가 true
    void addNMSAT(const vector<int> &as, const vector<int>
    &bs) { for (auto &a : as) for (auto &b : bs) add2SAT(a,
    b); } // (a1 and a2 and ... and aN) or (b1 and b2 and ...
    and bM)가 true
    void addAtLeastTwo(int a, int b, int c) { add2SAT(a, b);
    add2SAT(b, c); add2SAT(c, a); } // a, b, c 중 2개 이상이
    true
    void addAtMostOne(const vector<int> &v) { // v[0], v[1],
    ..., v[n - 1] 중 최대 하나만 true
        if (v.size() <= 1) return;
        adj.resize(2 * (n + v.size()));
        for (int i = 0; i < v.size(); i++) add2SAT(-v[i], n +
        1 + i);
        for (int i = 1; i < v.size(); i++) {
            int cur = n + 1 + i;
            int prv = n + i;
            add2SAT(-prv, cur);
            add2SAT(-prv, -v[i]);
        }
        n += v.size();
    }
}

```

```

void findSCC() {
    sccn.assign(2 * n, -1); // addAtMostOne호출하면 n
    // 증가하기 때문에 findSCC()할 때마다 2*n+1로 설정해줘야 함
    vector<int> dfsn(2 * n, 0);
    vector<int> s(2 * n);
    int top = 0, dfsi = 0, scci = 0;
    function<int(int)> dfs = [&](int cur) -> int {
        int low = dfsn[cur] = ++dfsi;
        s[top++] = cur;
        for (auto nxt : adj[cur]) if (!~sccn[nxt]) low =
            min(low, dfsn[nxt] ? dfsn[nxt] : dfs(nxt));
        if (low == dfsn[cur]) {
            do { sccn[s[--top]] = scci; } while (s[top] !=
                cur);
            ++scci;
        }
        return low;
    };
    for (int i = 0; i < 2 * n; i++) if (!dfsn[i]) dfs(i);
}
bool solve() { // 0(V + E) // V는 변수개수, E는 절 개수
    findSCC();
    for (int i = 1; i <= n; i++) if (sccn[f(i)] ==
        sccn[f(-i)]) return false;
    return true;
}
bool getSolution(int i) { // 1-based
    return sccn[f(-i)] > sccn[f(i)]; // sccn뒤집어져있어서
}
};
// false/true는 -/+로 표현
// graph.add2SAT(-1, 2); // (!x1 or x2)
// graph.add2SAT(-2, -3); // (!x2 or !x3)

```

4.9 Euler Circuit

```

// 양방향
struct Edge {
    int to, rev, cnt;
};

auto dfs = [&](auto &&dfs, int cur) -> void {
    while (!adj[cur].empty()) {
        auto &edge = adj[cur].back();
        if (edge.cnt == 0) adj[cur].pop_back();
        else {
            --edge.cnt;
            --adj[edge.to][edge.rev].cnt;
            int tmp = edge.idx;
            dfs(dfs, edge.to);
            edges.push_back(tmp);
        }
    }
    nodes.push_back(cur);
};

// 양방향 그래프
struct Edge { int to, rev, cnt; };
pair<bool, vector<int>> getCircuit(vector<vector<Edge>> adj,
int start) { // adj비워져도 괜찮으면 참조 사용 // 0(V+E)
    int E = 0;
    for (auto &r : adj) for (auto edge : r) E += edge.cnt;
    E >>= 1;
    vector<int> res;
    auto dfs = [&](auto &&dfs, int cur) -> void {
        while (!adj[cur].empty()) {
            auto &edge = adj[cur].back();
            if (edge.cnt == 0) adj[cur].pop_back();
            else {
                --edge.cnt;
                --adj[edge.to][edge.rev].cnt;
                dfs(dfs, edge.to);
            }
        }
        res.push_back(cur);
    };
    dfs(dfs, start);
    bool possible = (res.size() == E + 1 && res[0] ==
        res.back());
    return {possible, res};
}

```

// 단방향 그래프

```

pair<bool, vector<int>> getCircuit(vector<vector<pair<int,
int>>> adj, int start) { // adj비워져도 괜찮으면 참조 사용 //
0(V+E)
    int E = 0;
    for (auto &r : adj) for (auto [nxt, cnt] : r) E += cnt;
    vector<int> res;
    auto dfs = [&](auto &&dfs, int cur) -> void {
        while (!adj[cur].empty()) {
            auto &[nxt, cnt] = adj[cur].back();
            int tmp = nxt; // dangling 방지
            if (--cnt == 0) adj[cur].pop_back();
            dfs(dfs, tmp);
        }
        res.push_back(cur);
    };
    dfs(dfs, start);
    reverse(res.begin(), res.end());
    bool possible = (res.size() == E + 1 && res[0] ==
        res.back());
    return {possible, res};
}

// 오일러 회로는 모든 간선을 지나는 게 목표지, 모든 정점을
// 지나는 게 목표가 아님
// 따라서 간선들끼리만 연결되어있으면 가능하고, start는 간선과
// 연결되어 있는 정점 아무거나 가능

// 오일러 경로는 홀수 차수 정점이 2개 or 0개여야 가능.
// 만약 2개라면 해당정점들을 cnt=1인 간선으로 잇고 회로를 찾은
// 뒤 해당간선만 제거하면 됨

// 경로 구하고 싶으면 indg < outdeg인 지점을 start로 호출하면
// 됨, 이 경우 res[0] != res.back()이 됨

```

4.10 C3, C4

```

void getC3(const vector<vector<int>> &adj) { //
    sum_E{min(deg(u), deg(v))} = 0(m sqrt(m))
    vector<vector<int>> dag(adj.size());
    for (int u = 0; u < adj.size(); u++) {
        for (auto v : adj[u]) {
            if (adj[u].size() < adj[v].size() ||
                (adj[u].size() == adj[v].size() && u < v))
                dag[u].push_back(v);
        }
    } // dag에서 차수는 전부 0(sqrt(m)) 이하
    vector<int> mark(adj.size(), -1);
    for (int i = 0; i < adj.size(); i++) {
        for (auto j : adj[i]) mark[j] = i;
        for (auto j : dag[i]) {
            for (auto k : dag[j]) {
                if (mark[k] == i) cout << i << " " << j << " "
                    << k << "\n";
            }
        }
    }
}

int getC4(const vector<vector<int>> &adj, int MOD) {
    auto cmp = [&](int u, int v) { // u < v
        return adj[u].size() < adj[v].size() || (adj[u].size()
            == adj[v].size() && u < v);
    };
    vector<vector<int>> dag(adj.size());
    for (int u = 0; u < adj.size(); u++) {
        for (auto v : adj[u]) if (cmp(u, v))
            dag[u].push_back(v);
    }
    int res = 0;
    vector<int> cnt(adj.size());
    for (int i = 0; i < adj.size(); i++) {
        for (auto j : adj[i]) {
            for (auto k : dag[j]) if (cmp(i, k)) {
                res += cnt[k];
                if (res >= MOD) res -= MOD;
                ++cnt[k];
            }
        }
        for (auto j : adj[i]) {
            for (auto k : dag[j]) if (cmp(i, k)) cnt[k] = 0;
        }
    }
    return res;
}

```

```

}
// C3에선 rank(i)<rank(j)<rank(k)이므로 dag->dag순 탐색
// C4에선 rank(j)<rank(j')<rank(k) && rank(i)<rank(k) 일뿐 i랑
j간의 대소관계는 모두 고려해야하므로 adj->dag순 탐색

```

4.11 PEO

```

pair<bool, vector<int>> getPEO(int n, const
vector<vector<int>>& adj) { // 0(V+E) // 1-based
    vector<int> label(n + 1), order;
    vector<bool> visited(n + 1, false);
    vector<vector<int>> buckets(n + 1);
    for (int i = 1; i <= n; i++) buckets[0].push_back(i);
    int mx = 0;
    while (order.size() < n) { // mcs
        while (buckets[mx].empty()) --mx;
        int cur = buckets[mx].back();
        buckets[mx].pop_back();
        if (visited[cur]) continue;
        visited[cur] = true;
        order.push_back(cur);
        for (auto nxt : adj[cur]) if (!visited[nxt]) {
            buckets[++label[nxt]].push_back(nxt);
            mx = max(mx, label[nxt]);
        }
    }
    reverse(order.begin(), order.end());
    // check peo
    vector<int> pos(n + 1), parent(n + 1, -1);
    for (int i = 0; i < n; i++) pos[order[i]] = i;
    vector<vector<int>> chd(n + 1);
    for (int u = 1; u <= n; u++) {
        int mn = n + 1, w = -1;
        for (auto v : adj[u]) if (pos[v] > pos[u] && mn >
pos[v]) mn = pos[v], w = v;
        if (~w) {
            parent[u] = w;
            chd[w].push_back(u);
        }
    }
    vector<int> tag(n + 1);
    for (auto u : order) {
        tag[u] = u;
        for (auto v : adj[u]) tag[v] = u;
        for (auto v : chd[u]) {
            for (auto w : adj[v]) {
                if (pos[w] > pos[v] && w != u) {
                    if (tag[w] != u) return {false, order};
                }
            }
        }
    }
    return {true, order};
} // {isChordal, peo} // chordal: 길이 40이상의 모든 simple
cycle이 chord를 포함

```

4.12 Bimatch

```

// addEdge(l, r) : adj[l].push_back(r); // 0<=l<n1, 0<=r<n2
auto bimatch(int n1, int n2, const vector<vector<int>>& adj) {
// 0(VE) // 0-based
    vector<int> ml(n1, -1), mr(n2, -1), vr(n2);
    int t = 1;
    auto dfs = [&](auto &&dfs, int l) -> bool {
        for (auto r : adj[l]) if (vr[r] != t) {
            vr[r] = t;
            if (!mr[r] || dfs(dfs, mr[r])) {
                ml[l] = r;
                mr[r] = l;
                return true;
            }
        }
        return false;
    };
    int res = 0;
    for (int l = 0; l < n1; ++l, ++t) res += dfs(dfs, l);
    return tuple{res, ml, mr};
}

auto get_mvc(int n1, int n2, const vector<vector<int>>& adj,
const vector<int>& ml, const vector<int>& mr) { // 0(V+E) //
0-based
    vector<bool> vl(n1), vr(n2);
    auto dfs = [&](auto &&dfs, int l) -> void {
        vl[l] = true;
        for (auto r : adj[l]) if (~mr[r] && !vr[r] &&
!vl[mr[r]]) {

```

```

            vr[r] = true;
            dfs(dfs, mr[r]);
        }
    };
    for (int l = 0; l < n1; l++) if (!ml[l]) dfs(dfs, l);
    vector<int> mvcL, mvcR;
    for (int l = 0; l < n1; l++) if (!vl[l])
mvcL.push_back(l);
    for (int r = 0; r < n2; r++) if (vr[r]) mvcR.push_back(r);
    return pair{mvcL, mvcR};
}

```

4.13 Hopcroft Karp

```

tuple<int, vector<int>, vector<int>> bimatch(int n1, int n2,
const vector<vector<int>>& adj) { // 0(E sqrt(V)) // 0-based
    vector<int> matchL(n1, -1), matchR(n2, -1), lvl(n1),
ptr(n1);
    auto bfs = [&]() -> bool {
        queue<int> q;
        for (int l = 0; l < n1; l++) {
            if (!matchL[l]) {
                lvl[l] = 0;
                q.push(l);
            }
            else lvl[l] = -1;
        }
        bool flag = false;
        while (!q.empty()) {
            int l = q.front();
            q.pop();
            for (auto r : adj[l]) {
                if (!matchR[r]) flag = true;
                else if (!lvl[matchR[r]]) {
                    lvl[matchR[r]] = lvl[l] + 1;
                    q.push(matchR[r]);
                }
            }
        }
        return flag;
    };
    auto dfs = [&](auto &&dfs, int l) -> bool {
        for (int &i = ptr[l]; i < adj[l].size(); i++) {
            int r = adj[l][i];
            if (!matchR[r] || lvl[matchR[r]] == lvl[l] + 1 &&
dfs(dfs, matchR[r])) {
                matchL[l] = r;
                matchR[r] = l;
                return true;
            }
        }
        return false;
    };
    int res = 0;
    while (bfs()) {
        fill(ptr.begin(), ptr.end(), 0);
        for (int l = 0; l < n1; l++) res += (!matchL[l] &&
dfs(dfs, l));
    }
    return {res, matchL, matchR};
}

```

4.14 Dinic

```

template <typename F>
struct Dinic {
    struct Edge {
        int to, rev;
        F c, oc;
        F flow() { return max(oc - c, F(0)); }
    };
    vector<int> lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {} // 0-based
    void addEdge(int a, int b, F c, F rcap = 0) { //
양방향이면 rcap=c로 호출
        if (a == b) return; // self-loop는 최대유량에 영향 X
        adj[a].push_back({b, adj[b].size(), c, c});
        adj[b].push_back({a, adj[a].size() - 1, rcap, rcap});
    }
    F dfs(int cur, int t, F mn) {
        if (cur == t || !mn) return mn;
        for (int &i = ptr[cur]; i < adj[cur].size(); i++) {
            auto &e = adj[cur][i];
            if (lvl[e.to] != lvl[cur] + 1) continue;

```

```

        if (F f = dfs(e.to, t, min(mn, e.c))) {
            e.c -= f, adj[e.to][e.rev].c += f;
            return f;
        }
    }
    return 0;
}

template <bool scaling=false> F maxFlow(int s, int t) { //
    0(V^2 E) // max(cap)이 큰 경우 scaling=true가 빠름
    F res = 0; q[0] = s;
    for (int i = scaling ? 0 : 30; i < 31; i++) do {
        lvl = ptr = vector<int>(q.size());
        int qs = 0, qe = lvl[s] = 1;
        while (qs < qe && !lvl[t]) {
            int cur = q[qs++];
            for (Edge &e : adj[cur]) if (!lvl[e.to] && e.c
                >> (30 - i)) {
                q[qe++] = e.to, lvl[e.to] = lvl[cur] + 1;
            }
        }
        while (F f = dfs(s, t, numeric_limits<F>::max()))
            res += f;
    } while (lvl[t]);
    return res;
}

bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

// vector<pair<int, int> ref;
// ref.emplace_back(u, graph.adj[u].size()); graph.addEdge(u,
v);
// auto [u, idx] = ref[i]; cout << graph.adj[u][idx].flow() <<
"\n";
// 0(min(Ef, V^2 E)), all cap = 1: 0(min(V^{2/3}, E^{1/2}))E
// dinic with scaling 0(VE log(max_cap))
// 근데 보통 그냥 dinic이 더 빠름, 오히려 max_cap이 클 때
dinic with scaling이 좋음
// 무한간선 많은 경우 proxysource -> source로 K 용량의 간선을
두고 계산하면 애초에 최대유량이 K만큼으로 제한되므로
오버플로우 방지 가능
// 정점분할 시 양방향이라면 addEdge(out(a), in(b), c),
addEdge(out(b), in(a), c)
// 정점분할 시 maxFlow(out(s), in(t))

```

4.15 MCMF

```

template <typename C, typename F>
struct Graph {
    struct Edge {
        int to, rev;
        F c, oc;
        C cost;
        int from;
    };
    vector<vector<Edge>> adj;
    C DIST_INF = numeric_limits<C>::max();
    Graph(int n) : adj(n) {} // 0-based
    void addEdge(int a, int b, C cost, F c) {
        adj[a].push_back({b, adj[b].size(), c, cost, a});
        adj[b].push_back({a, adj[a].size() - 1, 0, 0, -cost,
b});
    }
    pair<C, F> mcmf(int s, int t, F targetFlow =
numeric_limits<F>::max()) { // 0(Ef) // average 0(Ef)
        C minCost = 0;
        F maxFlow = 0;
        vector<Edge*> pedge(adj.size());
        vector<C> dist(adj.size());
        vector<bool> inq(adj.size());
        while (maxFlow < targetFlow) {
            fill(dist.begin(), dist.end(), DIST_INF);
            dist[s] = 0;
            queue<int> q;
            q.push(s);
            inq[s] = true;
            while (!q.empty()) {
                int cur = q.front();
                q.pop();
                inq[cur] = false;
                for (auto &e : adj[cur]) if (e.c && dist[e.to]
                    > dist[cur] + e.cost) {
                    dist[e.to] = dist[cur] + e.cost;
                    pedge[e.to] = &e;
                    if (!inq[e.to]) {

```

```

                        q.push(e.to);
                        inq[e.to] = true;
                    }
                }
            }
            if (dist[t] == DIST_INF) break;
            F f = targetFlow - maxFlow;
            for (Edge *e = pedge[t]; e; e = pedge[e->from]) f
                = min(f, e->c);
            for (Edge *e = pedge[t]; e; e = pedge[e->from]) {
                e->c -= f;
                adj[e->to][e->rev].c += f;
            }
            minCost += f * dist[t];
            maxFlow += f;
        }
        return {minCost, maxFlow};
    }
};

```

4.16 MCMF(dijkstra)

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
template <typename C, typename F>
struct Graph {
    struct edge {
        int to, rev;
        F c, oc;
        C cost;
        int from;
    };
    int n;
    vector<vector<edge>> adj;
    vector<int> seen;
    vector<C> dist, pi;
    vector<edge*> pedge;
    const C DIST_INF = numeric_limits<C>::max() / 4;
    Graph(int n) : n(n), adj(n), seen(n), dist(n), pi(n),
pedge(n) {} // 0-based
    void addEdge(int a, int b, C cost, F c) {
        adj[a].push_back({b, adj[b].size(), c, cost, a});
        adj[b].push_back({a, adj[a].size() - 1, 0, 0, -cost,
b});
    }
    void path(int s) {
        fill(seen.begin(), seen.end(), 0);
        fill(dist.begin(), dist.end(), DIST_INF);
        dist[s] = 0;
        using elem = pair<C, int>;
        using pq_t = __gnu_pbds::priority_queue<elem,
greater<elem>>;
        pq_t q;
        vector<typename pq_t::point_iterator> its(n);
        q.push({0, s});
        while (!q.empty()) {
            int cur = q.top().second;
            q.pop();
            seen[cur] = 1;
            C di = dist[cur] + pi[cur];
            for (auto &e : adj[cur]) if (e.c && !seen[e.to]) {
                C val = di - pi[e.to] + e.cost;
                if (dist[e.to] > val) {
                    dist[e.to] = val;
                    pedge[e.to] = &e;
                    if (its[e.to] == q.end()) its[e.to] =
q.push({dist[e.to], e.to});
                    else q.modify(its[e.to], {dist[e.to],
e.to});
                }
            }
        }
        for (int i = 0; i < n; i++) pi[i] = min(pi[i] +
dist[i], DIST_INF);
    }
    pair<C, F> mcmf(int s, int t) {
        C minCost = 0;
        F maxFlow = 0;
        while (path(s), seen[t]) {
            F f = numeric_limits<F>::max();
            for (edge *e = pedge[t]; e; e = pedge[e->from]) f
                = min(f, e->c);
            for (edge *e = pedge[t]; e; e = pedge[e->from]) {

```

```

        e->c -= f;
        adj[e->to][e->rev].c += f;
    }
    maxFlow += f;
    minCost += f * (pi[t] - pi[s]);
}
return {minCost, maxFlow};
}
void setpi(int s) { // 음수비용 존재 시 mcmf전에 호출필요
// 0(VE)
    fill(pi.begin(), pi.end(), DIST_INF);
    pi[s] = 0;
    int it = n, ch = 1; ll v;
    while (ch-- && it--)
        for (int i = 0; i < n; i++) if (pi[i] != DIST_INF)
            for (edge& e : adj[i]) if (e.c)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // 비용 음수사이클 존재
}
};

```

4.17 LCA

```

// 1-based (ac default값이 0이라서 0-based안됨)
int mx = __lg(n);
vector<int> dep(n + 1);
vector<ac(n + 1, vector<int>(mx + 1));
auto dfs = [&](auto &&dfs, int cur, int par) -> void {
    for (auto nxt : adj[cur]) if (nxt != par) {
        dep[nxt] = dep[cur] + 1;
        ac[nxt][0] = cur;
        for (int i = 1; i <= mx; i++) ac[nxt][i] =
            ac[ac[nxt][i - 1]][i - 1];
        dfs(dfs, nxt, cur);
    }
};
dfs(dfs, 1, -1); // 0(NlogN)
auto getLCA = [&](int a, int b) { // 0(logN)
    if (dep[a] > dep[b]) swap(a, b);
    for (int diff = dep[b] - dep[a]; diff; diff &= diff - 1) b
        = ac[b][__builtin_ctz(diff)];
    if (a == b) return a;
    for (int i = mx; i >= 0; i--) if (ac[a][i] != ac[b][i]) a
        = ac[a][i], b = ac[b][i];
    return ac[a][0];
};
auto getLCA2 = [&](int a, int b, int root) {
    int x = getLCA(root, a);
    int y = getLCA(root, b);
    int res = getLCA(a, b);
    if (dep[res] < dep[x]) res = x;
    if (dep[res] < dep[y]) res = y;
    return res;
};

```

4.18 LCA(RMQ)

```

vector<int> dep(n + 1), pos(n + 1), euler(2 * n - 1);
int ei = 0;
auto mkt = [&](auto &&mkt, int cur, int par) -> void {
    euler[pos[cur] = ei++] = cur;
    for (auto nxt : adj[cur]) if (nxt != par) {
        dep[nxt] = dep[cur] + 1;
        mkt(mkt, nxt, cur);
        euler[ei++] = cur;
    }
};
mkt(mkt, 1, -1);
vector<vector<int>> ac(__lg(ei) + 1, euler);
for (int i = 1; i <= __lg(ei); i++) {
    for (int j = 0; j + (1 << i) - 1 < ei; j++) {
        int u = ac[i - 1][j];
        int v = ac[i - 1][j + (1 << i) - 1];
        ac[i][j] = dep[u] < dep[v] ? u : v;
    }
}
auto getLCA = [&](int a, int b) {
    auto [l, r] = minmax(pos[a], pos[b]);
    int i = __lg(r - l + 1);
    int u = ac[i][l];
    int v = ac[i][r - (1 << i) + 1];
    return dep[u] < dep[v] ? u : v;
};

```

4.19 ETT

```

// root->v path add / node v query
// update(in[v], out[v], add) / query(in[v], out[v])
// node v add / root->v path query
// update(in[v], out[v], add) / query(in[v])
// 루트가 r로 바뀌었을 때 v의 subtree
// v == r : 전체 트리
// v가 r의 조상이 아님 : 기존 subtree(v) = [tin[v], tout[v]]
// v가 r의 조상임 : 전체에서, v 아래에서 r로 가는 child의
subtree를 뺀 것

```

4.20 HLD

```

vector<vector<int>> chd(n);
vector<int> sz(n, 1), dep(n), p(n), in(n), top(n);
auto mkt = [&](auto &&mkt, int cur, int par) -> void {
    for (auto nxt : adj[cur]) if (nxt != par) {
        p[nxt] = cur;
        dep[nxt] = dep[cur] + 1;
        mkt(mkt, nxt, cur);
        chd[cur].push_back(nxt);
        sz[cur] += sz[nxt];
        if (sz[chd[cur][0]] < sz[nxt]) swap(chd[cur][0],
            chd[cur].back());
    }
};
mkt(mkt, root, -1);
int dfsi = 0;
auto ett = [&](auto &&ett, int cur) -> void {
    in[cur] = ++dfsi;
    for (auto nxt : chd[cur]) {
        top[nxt] = (nxt == chd[cur][0] ? top[cur] : nxt);
        ett(ett, nxt);
    }
};
top[root] = root;
ett(ett, root);
segtree seg(n);
auto query = [&](int a, int b) { // 0(log^2N)
    int res = 0;
    while (top[a] != top[b]) {
        if (dep[top[a]] > dep[top[b]]) swap(a, b);
        res += seg.query(in[top[b]], in[b]);
        b = p[top[b]];
    }
    if (dep[a] > dep[b]) swap(a, b);
    res += seg.query(in[a], in[b]); // 정점쿼리
    // if (in[a] + 1 <= in[b]) res += seg.query(in[a] + 1,
    in[b]); // 간선쿼리
    return res;
};
// seg.update(in[a], b);
// ett: [in[cur], in[cur] + sz[cur] - 1]

```

4.21 Centroid Decomposition

```

vector<int> sz(n + 1);
vector<bool> rm(n + 1);
auto get_sz = [&](auto &&get_sz, int cur, int par) -> int {
    sz[cur] = 1;
    for (auto nxt : adj[cur]) if (nxt != par && !rm[nxt]) {
        sz[cur] += get_sz(get_sz, nxt, cur);
    }
    return sz[cur];
};
auto get_ct = [&](auto &&get_ct, int cur, int par, int tot) ->
int {
    for (auto nxt : adj[cur]) if (nxt != par && !rm[nxt]) {
        if (sz[nxt] * 2 > tot) return get_ct(get_ct, nxt, cur,
            tot);
    }
    return cur;
};
vector<int> a(n + 1);
vector<int> dirty;
auto f = [&](auto &&f, int cur, int par) -> ll {
    ll res = 0;
    // res <- cur
    for (auto nxt : adj[cur]) if (nxt != par && !rm[nxt]) {
        res += f(f, nxt, cur);
    }
    return res;
};
auto update = [&](auto &&update, int cur, int par) -> void {

```

```

// a[cur];
// dirty.push_back(cur);
for (auto nxt : adj[cur]) if (nxt != par && !rm[nxt]) {
    update(update, nxt, cur);
}
};
auto dnc = [&](auto &&dnc, int cur) -> ll {
    int tot = get_sz(get_sz, cur, -1);
    int ct = get_ct(get_ct, cur, -1, tot);
    rm[ct] = 1;
    ll res = 0;
    // centroid가 끝점인 경로도 세야됨
    for (auto nxt : adj[ct]) if (!rm[nxt]) {
        res += f(f, nxt, -1);
        update(update, nxt, -1);
    }
    for (auto x : dirty) a[x] = 0;
    dirty.clear();
    for (auto nxt : adj[ct]) if (!rm[nxt]) {
        res += dnc(dnc, nxt);
    }
    return res;
};

```

4.22 Virtual Tree

```

// compress
sort(nodes.begin(), nodes.end(), [&](int a, int b) {
    return dfsn[a] < dfsn[b];
});
int tmp = nodes.size();
for (int i = 1; i < tmp; i++) nodes.push_back(getLCA(nodes[i - 1], nodes[i]));
sort(nodes.begin(), nodes.end(), [&](int a, int b) {
    return dfsn[a] < dfsn[b];
});
nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
root = nodes[0];
for (int i = 1; i < nodes.size(); i++) {
    int u = getLCA(nodes[i - 1], nodes[i]);
    int v = nodes[i];
    chd[u].emplace_back(v, dist[v] - dist[u]);
}

// clean-up // O(sum(nodes.size()))
for (auto x : nodes) {
    chd[x].clear();
}

```

4.23 Graph Theory

트리의 중심

모든 트리는 1개 or 2개의 중심을 가지며 2개라면 그 두 정점은 반드시 인접함
 트리의 모든 지름은 반드시 트리의 중심을 지남(=공유함)
 트리의 지름이 짝수라면: 중심은 1개, 지름의 중간지점이 중심
 홀수라면: 중심은 2개, 지름의 가운데 간선으로 연결된 두 정점이 중심
 즉, 중심이 1개라면 모든 지름은 그 중심을 지나고 2개라면 모든 지름은 그 두 중심을 잇는 간선을 지남
 트리의 모든 지름이 공유하는 정점들의 집합은 항상 하나의 연결된 경로를 이룸

평면그래프

평면 그래프에서 $v - e + f = 1 + c$ (v : 정점, e : 간선, f : 면, c : 연결요소 수)
 연결된 평면 그래프에서 $v \geq 3$ 인 중복 간선없는 단순 평면 그래프는 $e \leq 3v - 6$ 을 만족
 $2e =$ 각 면에서 변의 개수의 합 $\geq 3f$ 이므로 $v - e + f = 2$ 와 연립하면 $e \leq 3v - 6$ (연결된 평면 그래프에서) 이분그래프라면 사이클의 최소 길이가 4이므로 $2e \geq 4f, e \leq 2v - 4$

매칭

Gallai's Identity: $|IS_{max}| + |VC_{min}| = |V|$

IS_{max} 계산은 일반그래프에선 NP-hard

König's theorem: 이분그래프에서 최대매칭 = $|VC_{min}|$

$IS_{max,L} = L - VC_{min,L}, IS_{max,R} = R - VC_{min,R}$

Dilworth's theorem: 부분 순서 집합에서 최대 반사율의 크기는 사슬 분할의 최소 개수와 같음

antichain은 v_{in} 이 $IS_{max,L}$ 이고 v_{out} 이 $IS_{max,R}$ 인 v 들을 모으면 됨

clique는 complement graph에서의 Independent set과 같음

hall's marriage theorem: 이분그래프 $G = (L + R, E)$ 에서 L 의 부분집합 S 에 대해 이와 연결된 R 의 부분집합을 $N(S)$ 라 할 때, 이 이분그래프에 perfect matching이 존재할 필요충분조건은 모든 S 에 대하여 $|S| \leq |N(S)|$
 $\rightarrow k$ -정규 이분 그래프는 항상 perfect matching을 가짐

5 Math

5.1 Linear Sieve

```

vector<int> getLpf(int n) {
    vector<int> primes, lpf(n + 1); // least prime factor // O(N)
    for (int i = 2; i <= n; i++) {
        if (!lpf[i]) {
            lpf[i] = i;
            primes.push_back(i);
        }
        for (auto p : primes) {
            if (i > n / p) break;
            lpf[i * p] = p;
            if (i % p == 0) break;
        }
    }
    return lpf;
}

```

5.2 Multiplicative Function

```

vector<ll> getMulFunc(int n) { // O(N)
    vector<ll> func(n + 1);
    func[1] = 1;
    vector<int> primes, lpf(n + 1), lpe(n + 1); // least prime factor, least prime exponent
    for (ll i = 2; i <= n; i++) {
        if (!lpf[i]) {
            lpf[i] = i;
            lpe[i] = 1;
            primes.push_back(i);
            // func[i] = ? // func(p) // i는 소수
            // ex) mu[i] = -1;
        }
        for (auto p : primes) {
            if (i * p > n) break;
            lpf[i * p] = p;
            if (i % p == 0) {
                lpe[i * p] = lpe[i] + 1;
                // func[i * p] = ? // func(i' * p * p) // 소인수 p의 지수 1 증가
                // ex) mu[i * p] = 0;
                break;
            }
            else {
                lpe[i * p] = 1;
                func[i * p] = func[i] * func[p]; // gcd(i, p) = 1
            }
        }
    }
    return func;
}

```

5.3 Burnside's Lemma

```

ll bracelet(int c, int n) { // 색깔 c종류 염주순열 개수
    vector<ll> powc(n + 1, 1);
    for (int i = 1; i <= n; i++) powc[i] = powc[i - 1] * c;
    ll a = powc[n / 2 + 1];
    ll b = powc[n / 2];
    ll ans = (n & 1) ? n * a : n / 2 * (a + b);
    for (int i = 0; i < n; i++) ans += powc[__gcd(i, n)];
    return ans / (2 * n);
}

```

5.4 Freivalds' Algorithm

```

template <typename T>
bool freivalds(const vector<vector<T>> &a, const vector<vector<T>> &b, const vector<vector<T>> &c) { // O(K N^2)
    int n = a.size();
    vector<unsigned long long> Bx(n), x(n);
    static Random<int> rand(0, 1);
    constexpr int K = 30;
    for (int it = K; it--;) {
        for (auto &e : x) e = rand();
        for (int i = 0; i < n; i++) {
            Bx[i] = 0;
            for (int j = 0; j < n; j++) Bx[i] += b[i][j] * x[j];
        }
        for (int i = 0; i < n; i++) {
            unsigned long long ABx_i = 0, Cx_i = 0;
            for (int j = 0; j < n; j++) {

```

```

        ABx_i += a[i][j] * Bx[j];
        Cx_i += c[i][j] * x[j];
    }
    if (ABx_i != Cx_i) return false;
}
}
return true;
}
// n*n행렬 A, B, C에 대해 AB==C(mod 2^64)확인
5.5 egcd
tuple<ll, ll> egcd(ll a, ll b) { // ax + by = gcd(a, b)
    if (b == 0) return {1, 0, a};
    auto [x, y, g] = egcd(b, a % b);
    return {y, x - (a / b) * y, g};
}
ll modInv(ll a, ll b) {
    auto [x, y, g] = egcd(a, b);
    return g != 1 ? -1 : (x + b) % b;
} // modInv(n, MOD)
vector<ll> inv(n + 1, 1);
for (int i = 2; i <= n; ++i) inv[i] = inv[p % i] * (p - p / i) % p;
5.6 FFT
// 4 * n * max_{k}(sum|a[i]*b[k-i]|) << 2^53 : double
namespace Poly { // FFT
template <typename real_t>
void fft(vector<complex<real_t>> &a, bool inv) {
    using cpx = complex<real_t>;
    int n = a.size(), L = _lg(n);
    static vector<complex<long double>> R(2, 1);
    static vector<cpx> rt(2, 1);
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        for (int i = k; i < 2 * k; i++) rt[i] = R[i] = i & 1 ?
            R[i/2] * x : R[i/2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                auto x = (real_t *)&rt[j+k], y = (real_t *)
                    &a[i+j+k];
                cpx z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
                // cpx z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
        }
    }
    if (inv) {
        reverse(a.begin() + 1, a.end());
        for (auto &e : a) e /= real_t(a.size());
    }
}
template <typename real_t, typename T>
vector<ll> multiply(const vector<T> &A, const vector<T> &B) {
    assert(!A.empty() && !B.empty());
    using cpx = complex<real_t>;
    int need = A.size() + B.size() - 1;
    int n = 1;
    while (n < need) n <= 1;
    vector<cpx> in(n), out(n);
    for (int i = 0; i < A.size(); i++) in[i] = A[i];
    for (int i = 0; i < B.size(); i++) in[i].imag(B[i]);
    fft(in, false);
    for (auto &x : in) x *= x;
    for (int i = 0; i < n; i++) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out, false);
    vector<ll> res(need);
    for (int i = 0; i < res.size(); i++) res[i] = llround(imag(out[i]) / (4 * n));
    return res;
}
}
}

```

```

namespace Poly { // 정확도 높은 FFT
template <typename real_t>
void fftPrecisely(vector<complex<real_t>> &a, bool inv) {
    using cpx = complex<real_t>;
    int n = a.size(), L = _lg(n);
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i >> 1] | ((i & 1) << L)) >> 1;
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
    vector<cpx> roots(n / 2);
    real_t ang = 2 * acos(real_t(-1)) / n * (inv ? -1 : 1);
    for (int i = 0; i < n / 2; i++) roots[i] = cpx(cos(ang * i), sin(ang * i));
    for (int m = 2; m <= n; m <= 1) {
        int step = n / m;
        for (int i = 0; i < n; i += m) {
            for (int j = 0; j < m / 2; j++) {
                cpx even = a[i + j];
                cpx odd = roots[step * j] * a[i + j + m / 2];
                a[i + j] = even + odd;
                a[i + j + m / 2] = even - odd;
            }
        }
    }
    if (inv) for (auto &e : a) e /= cpx(n);
}
static constexpr int SPLIT_BIT = 15;
static constexpr ll SPLIT_BASE = 1LL << SPLIT_BIT;
static constexpr ll SPLIT_BASE_SQ = 1LL << SPLIT_BIT * 2;
static constexpr int SPLIT_MASK = (1 << SPLIT_BIT) - 1;
template <typename real_t=double, typename T>
vector<ll> multiplyPrecisely(const vector<T> &A, const vector<T> &B) {
    assert(!A.empty() && !B.empty());
    using cpx = complex<real_t>;
    int need = A.size() + B.size() - 1;
    int n = 1;
    while (n < need) n <= 1;
    vector<cpx> a(n), b(n), c1(n), c2(n);
    // for (int i = 0; i < A.size(); i++) a[i] = cpx(A[i] >> SPLIT_BIT, A[i] & SPLIT_MASK); // A[i], B[i] > 0
    // for (int i = 0; i < B.size(); i++) b[i] = cpx(B[i] >> SPLIT_BIT, B[i] & SPLIT_MASK);
    for (int i = 0; i < A.size(); i++) a[i] = cpx(A[i] / SPLIT_BASE, A[i] % SPLIT_BASE);
    for (int i = 0; i < B.size(); i++) b[i] = cpx(B[i] / SPLIT_BASE, B[i] % SPLIT_BASE);
    fftPrecisely(a, false);
    fftPrecisely(b, false);
    for (int i = 0; i < n; i++) {
        int j = (i ? (n - i) : i);
        cpx ans1 = (a[i] + conj(a[j])) * cpx(0.5, 0);
        cpx ans2 = (a[i] - conj(a[j])) * cpx(0, -0.5);
        cpx ans3 = (b[i] + conj(b[j])) * cpx(0.5, 0);
        cpx ans4 = (b[i] - conj(b[j])) * cpx(0, -0.5);
        c1[i] = (ans1 * ans3) + (ans1 * ans4) * cpx(0, 1);
        c2[i] = (ans2 * ans3) + (ans2 * ans4) * cpx(0, 1);
    }
    fftPrecisely(c1, true);
    fftPrecisely(c2, true);
    vector<ll> res(need);
    for (int i = 0; i < res.size(); i++) {
        ll av = llround(c1[i].real());
        ll bv = llround(c1[i].imag()) + llround(c2[i].real());
        ll cv = llround(c2[i].imag());
        // res[i] = (av << (SPLIT_BIT * 2)) + (bv << SPLIT_BIT) + cv; // av, bv, cv > 0
        res[i] = av * SPLIT_BASE_SQ + bv * SPLIT_BASE + cv;
    }
    return res;
}
}
// multiply<double_t>(f, g) : fft 다항식 곱셈 O(N logN)
// multiplyPrecisely<double_t>(f, g) : 정확도 높은 fft 다항식 곱셈 O(N logN)
5.7 FFT Division
namespace Poly { // $(K^2 \log\{N\})$ 키타마사
template <typename T>

```

```

vector<ll> divideModNaive(ll n, const vector<T> &g, const ll
MOD) { // f(x)=x^n에 대해 f % g 계산 // g 최고차항 계수는 1 //
g차수 k일 때 O(K^2 logN)
    const int k = g.size() - 1;
    assert(g[k] == 1);
    vector<ll> res(k, xn(k));
    res[0] = 1;
    if (k == 1) {
        xn[0] = (MOD - g[0]) % MOD; // x === -g[0] (mod x +
g[0])
        if (xn[0] < 0) xn[0] += MOD;
    }
    else xn[1] = 1; // x
    auto add = [&MOD](ll &a, ll b) { a = (a + b) % MOD; };
    auto sub = [&MOD](ll &a, ll b) { a = (a - b) % MOD; if (a
< 0) a += MOD; };
    auto mul = [&](vector<ll> &a, const vector<ll> &b) {
        vector<ll> scratch(2 * k - 1);
        for (int i = 0; i < k; i++) if (a[i]) for (int j = 0;
j < k; ++j) add(scratch[i + j], a[i] * b[j]); //
scratch = a * b
        for (int i = 2 * k - 2; i >= k; i--) if (scratch[i])
        for (int j = 1; j <= k; ++j) sub(scratch[i - j],
scratch[i] * g[k - j]); // scratch %= g
        for (int i = 0; i < k; i++) a[i] = scratch[i];
    };
    while (n) {
        if (n & 1) mul(res, xn);
        mul(xn, xn);
        n >>= 1;
    }
    return res;
}

template <typename T>
ll kitamasaNaive(ll n, const vector<T> &a, const vector<T>
&coef, const ll MOD) { // a는 초항{a1, a2, ..., ak}, c는
계수{c1, c2, ..., ck} // A_n = (C_1 x A_{n-1} + C_2 x A_{n-2} +
... + C_k x A_{n-k}) mod M일 때 A_n 계산
    assert(n >= 1 && !coef.empty() && a.size() ==
coef.size());
    if (n <= a.size()) return (a[n - 1] % MOD + MOD) % MOD;
    vector<ll> f(coef.size() + 1, 1); // x^k - c_1 x^(k-1) -
c_2 x^(k-2) - ... - c_k x^0
    for (int i = 0; i < coef.size(); i++) f[coef.size() - 1 -
i] = (MOD - coef[i]) % MOD;
    auto d = divideModNaive(n - 1, f, MOD); // x^(n-1)을 {x^k
- c_1 x^(k-1) - c_2 x^(k-2) - ... - c_k x^0}로 나눈
나머지와 같음
    ll res = 0; // an = a1 * d1 + a2 * d2 + ... + ak * dk
    for (int i = 0; i < d.size(); i++) res = (res + a[i] *
d[i]) % MOD;
    return res;
}

namespace Poly { // NTT 다항식 나눗셈, 키타마사 // $O(K
\log{K} \log{N})$ 키타마사
template <ll p, ll primitiveRoot, typename T>
vector<ll> invertMod(const vector<T> &f, int deg) { // f(x)^-1
mod x^deg 계산 // 계수는 mod p로 계산
    assert(!f.empty() && f[0] % p != 0); // f(x)의 상수항이
0이면 역원이 존재하지 않음 // 참고로 divideMod(f, g)에서
g의 계수를 뒤집은 게 invertMod의 f이므로 f[0]!=0이란 건
divideMod에서 g의 최고차항의 계수가 0이 아닌란 것과 동치임
    vector<ll> res = {mint<p>(f[0] % p).inv().value}; //
f(x)^-1 mod x^1
    for (int curDeg = 1; curDeg < deg; curDeg <= 1) {
        int nxtDeg = curDeg << 1;

        // prod = f(x) * res(x)
        vector<ll> prod = multiplyMod<p, primitiveRoot>(res,
vector<ll>(f.begin(), f.begin() + min(int(f.size()),
nxtDeg)));
        prod.resize(nxtDeg);

        // prod = 2 - f(x) * res(x)
        if ((prod[0] - 2) < 0) prod[0] += p;
        for (int i = 0; i < prod.size(); i++) if (prod[i] !=
0) prod[i] = p - prod[i];

        // 새로운 res(x) = res(x) * (2 - f(x) * res(x))

```

```

        res = multiplyMod<p, primitiveRoot>(res, prod);
        res.resize(nxtDeg);
    }

    res.resize(deg);
    return res;
}

template <ll p, ll primitiveRoot, typename T>
pair<vector<ll>, vector<ll>> divideMod(const vector<T> &f,
const vector<T> &g) {
    int n = f.size(), m = g.size();
    if (n < m) {
        vector<ll> res(n);
        for (int i = 0; i < n; i++) {
            res[i] = f[i] % p;
            if (res[i] < 0) res[i] += p;
        }
        return {vector<ll>(0), res};
    }

    vector<ll> F(f.rbegin(), f.rend());
    vector<ll> G(g.rbegin(), g.rend());

    vector<ll> q = multiplyMod<p, primitiveRoot>(F,
invertMod<p, primitiveRoot>(G, n - m + 1));
    q.resize(n - m + 1);
    reverse(q.begin(), q.end());

    // r(x) = f(x) - q(x) * g(x)
    vector<ll> r = multiplyMod<p, primitiveRoot>(q, g);
    r.resize(m - 1);

    for (int i = 0; i < m - 1; i++) {
        r[i] = (f[i] - r[i]) % p;
        if (r[i] < 0) r[i] += p;
    }

    return {q, r};
}

template <ll p, ll primitiveRoot, typename T>
vector<ll> remainderMod(const vector<T> &f, const vector<T>
&g) {
    return divideMod<p, primitiveRoot>(f, g).second;
}

template <ll p, ll primitiveRoot, typename T>
vector<ll> divideMod(ll n, const vector<T> &g) { // f(x)=x^n에
대해 f % g 계산 // g차수 k일 때 O(K logK logN)
    vector<ll> res = {1};
    vector<ll> xn = {0, 1};
    while (n) {
        if (n & 1) res = remainderMod<p,
primitiveRoot>(multiplyMod<p, primitiveRoot>(res, xn),
g);
        xn = remainderMod<p, primitiveRoot>(multiplyMod<p,
primitiveRoot>(xn, xn), g);
        n >>= 1;
    }
    return res;
}

template <ll p, ll primitiveRoot, typename T> // p = a * 2^b +
1 꼴 소수
ll kitamasaNTT(ll n, const vector<T> &a, const vector<T>
&coef) { // a는 초항{a1, a2, ..., ak}, c는 계수{c1, c2, ...,
ck} // A_n = (C_1 x A_{n-1} + C_2 x A_{n-2} + ... + C_k x A_{n-k})
mod P일 때 A_n 계산
    assert(n >= 1 && !coef.empty() && a.size() ==
coef.size());
    if (n <= a.size()) return a[n - 1];
    vector<ll> f(coef.size() + 1, 1); // x^k - c_1 x^(k-1) -
c_2 x^(k-2) - ... - c_k x^0
    for (int i = 0; i < coef.size(); i++) f[coef.size() - 1 -
i] = -coef[i];
    auto d = divideMod<p, primitiveRoot>(n - 1, f); //
x^(n-1)을 {x^k - c_1 x^(k-1) - c_2 x^(k-2) - ... - c_k
x^0}로 나눈 나머지
    ll res = 0; // an = a1 * d1 + a2 * d2 + ... + ak * dk
    for (int i = 0; i < d.size(); i++) res = (res + a[i] *
d[i]) % p;

```

```

    return res;
}
}
/*
multiplyMod<p, primitiveRoot>(f, g) : ntt 다항식 곱셈 O(N
logN)
invertMod<p, primitiveRoot>(f, deg) :  $f(x)^{-1}$ 
 $\pmod{x^{\deg}}$  계산 |  $O(N \log N)$  |
divideMod<p, primitiveRoot>(f, g) :  $f(x) / g(x)$ ,
 $f(x) \pmod{g(x)}$  계산 |  $O(N \log N)$  |
divideMod<p, primitiveRoot>(n, g) :  $x^n \pmod{g(x)}$  계산 |
 $O(K \log K)$  |  $g(x)$ 의 최고차항의 차수
divideModNaive(n, g, MOD) :  $x^n \pmod{g(x)}$  계산 |  $O(K^2 \log N)$  |
 $g(x)$ 의 최고차항의 차수
kitamasaNTT<p, primitiveRoot>(n, a, coef) : 초항  $a_1, \dots$ ,
 $a_k$ 와 점화식  $a_i = c_1 a_{i-1} + \dots + c_k a_{i-k}$ 에 대해
 $a_n \pmod{p}$  계산 |  $O(K \log K \log N)$  |  $g(x)$ 의
최고차항의 차수
kitamasaNaive(n, a, coef, MOD) : 초항  $a_1, \dots, a_k$ 과
점화식  $a_i = c_1 a_{i-1} + \dots + c_k a_{i-k}$ 에 대해  $a_n$ 
 $\pmod{MOD}$  계산 |  $O(K^2 \log N)$  |  $g(x)$ 의
최고차항의 차수
*/

```

5.8 Multipoint Evaluation

```

namespace Poly { // multipoint evaluation
template <ll p, ll primitiveRoot, typename T>
vector<ll> multiEval(const vector<T> &f, const vector<T>
&qs) { //  $O(N \log N + q \log^2 q)$ 
    if (qs.empty()) return {};
    assert(!f.empty());
    vector<vector<ll>> tree(4 * qs.size());
    auto build = [&](auto &&build, int node, int s, int e)
    {
        if (s == e) {
            tree[node] = {(p - qs[s] % p) % p, 1};
            return;
        }
        int m = s + e >> 1;
        build(build, node << 1, s, m);
        build(build, node << 1 | 1, m + 1, e);
        tree[node] = multiplyMod<p,
            primitiveRoot>(tree[node << 1], tree[node << 1 |
            1]);
    };
    build(build, 1, 0, qs.size() - 1); //  $O(q \log^2 q)$ 
    vector<ll> res(qs.size());
    auto eval = [&](auto &&eval, vector<ll> poly, int
    node, int s, int e) {
        poly = remainderMod<p, primitiveRoot>(poly,
            tree[node]);
        if (s == e) {
            assert(!poly.empty());
            res[s] = poly[0];
            return;
        }
        int m = s + e >> 1;
        eval(eval, poly, node << 1, s, m);
        eval(eval, poly, node << 1 | 1, m + 1, e);
    };
    eval(eval, f, 1, 0, qs.size() - 1);
    return res;
}
}

```

5.9 NTT

```

namespace Poly { // NTT
template <ll p> ll binpow(ll a, ll n) { //
(MOD-1)2 <= INT64_MAX
    ll res = 1;
    for (; n; n >>= 1) {
        if (n & 1) res = res * a % p;
        a = a * a % p;
    }
    return res;
}
template <ll p, ll primitiveRoot>
void ntt(vector<ll> &a, bool inv) {
    int n = a.size(), L = __lg(n);
    static vector<ll> rt(2, 1);
    for (static int k = 2, s = 2; k < n; k <= 1, ++s) {
        rt.resize(n);
        ll z[2] = {1, binpow<p>(primitiveRoot, p >> s)};

```

```

        for (int i = k; i < 2 * k; i++) rt[i] = rt[i >> 1] *
            z[i & 1] % p;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i >> 1] | ((i &
        1) << L) >> 1);
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i],
        a[rev[i]]);
    for (int k = 1; k < n; k <= 1) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                ll u = a[i + j], v = a[i + j + k] * rt[j + k]
                    % p;
                a[i + j] = u + v < p ? u + v : u + v - p;
                a[i + j + k] = u - v >= 0 ? u - v : u - v + p;
            }
        }
    }
    if (inv) {
        reverse(a.begin() + 1, a.end());
        ll invN = binpow<p>(n, p - 2);
        for (auto &e : a) e = e * invN % p;
    }
}

```

```

template <ll p, ll primitiveRoot, typename T>
vector<ll> multiplyMod(const vector<T> &A, const vector<T> &B)
{
    assert(!A.empty() && !B.empty());
    int need = A.size() + B.size() - 1;
    int n = 1;
    while (n < need) n <= 1;
    assert(n <= ((p - 1) & -(p - 1))); // assert(n <= 2b)
    vector<ll> a(n), b(n);
    for (int i = 0; i < A.size(); i++) {
        a[i] = A[i] % p;
        if (a[i] < 0) a[i] += p;
    }
    for (int i = 0; i < B.size(); i++) {
        b[i] = B[i] % p;
        if (b[i] < 0) b[i] += p;
    }
    ntt<p, primitiveRoot>(a, false);
    ntt<p, primitiveRoot>(b, false);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % p;
    ntt<p, primitiveRoot>(a, true);
    a.resize(need);
    return a;
}

```

$p = a \cdot 2^b + 1$	a	b	2^b	g
104'857'601	25	22	4194304	3
167'772'161	5	25	33554432	3
469'762'049	7	26	67108864	3
998'244'353	119	23	8388608	3
1'004'535'809	479	21	2097152	3
1'012'924'417	483	21	2097152	5
2'281'701'377	17	27	134217728	3
2'483'027'969	37	26	67108864	3

5.10 CRT

```

pll merge(const pll &a, const pll &b) {
    auto [p1, m1] = a;
    auto [p2, m2] = b;
    ll g = __gcd(p1, p2);
    if ((m2 - m1) % g) return {-1, -1};
    ll x = (m2 - m1) / g * modInverse(p1 / g, p2 / g) % (p2 /
        g);
    return {p1 / g * p2, p1 * x + m1};
}
pll crt(const vector<pll> &rems) { // rems 원소는 {p, n % p} 꼴
//  $O(N \log P)$  ( $P = p_1 p_2 \dots p_n$ )
    pll res(1, 0);
    for (auto &p : rems) {
        res = merge(res, p);
        if (res.first == -1) return {-1, -1};
    }
    if (res.second < 0) res.second += res.first;
    return res;
}
// crt().first == -1이면 연립합동식에 해 존재
5.11 Miller-Rabin
inline ll multiply(ll a, ll b, ll mod) { return __int128(a) *
    b % mod; }

```

```

11 power(ll a, ll n, ll mod) { //a ^ n % mod
    ll res = 1;
    while (n) {
        if (n & 1) res = multiply(res, a, mod);
        a = multiply(a, a, mod);
        n >>= 1;
    }
    return res;
}

bool isPrime(ll n) { // 밀러-라빈, O(7log^3N)
    if (n <= 1) return false;
    ll d = n - 1, r = 0;
    while (~d & 1) d >>= 1, ++r;
    auto check = [&](ll a) {
        ll remain = power(a, d, n);
        if (remain == 1 || remain == n - 1) return true;
        for (int i = 1; i < r; i++) {
            remain = multiply(remain, remain, n);
            if (remain == n - 1) return true;
        }
        return false;
    };
    vector<ll> nums = {2, 325, 9375, 28178, 450775, 9780504,
1795265022}; // ull
    // int 범위는 {2, 7, 61}
    for (ll a : nums) if (a % n && !check(a)) return false;
    return true;
}

5.12 Pollard's rho
namespace PollardRho {
    ll getPrime(ll n) {
        if (~n & 1) return 2;
        if (isPrime(n)) return n;
        ll a, b, c, g;
        auto f = [&c, &n](ll x) { return (multiply(x, x, n) + c) %
n; };
        do {
            a = b = rand() % (n - 2) + 2;
            c = rand() % 10 + 1;
            do {
                a = f(a);
                b = f(f(b));
                g = __gcd(abs(a - b), n);
            } while (g == 1);
        } while (g == n);
        return getPrime(g);
    }

    vector<pair<ll, ll>> factorize(ll n) { // O(\sqrt[4]{N})
        assert(n > 1);
        vector<pair<ll, ll>> primes;
        while (n > 1) {
            ll prime = getPrime(n), cnt = 0;
            while (n % prime == 0) n /= prime, ++cnt;
            primes.push_back({prime, cnt});
        }
        sort(primes.begin(), primes.end());
        return primes;
    }

    vector<ll> getAllDivisors(ll n) {
        vector<ll> res = {1};
        if (n == 1) return res;
        for (auto [p, e] : factorize(n)) {
            int sz = res.size();
            ll curP = p;
            for (int _ = e; _--;) {
                for (int i = 0; i < sz; i++) res.push_back(res[i] *
curP);
                curP *= p;
            }
        }
        assert(false, "정렬 필요한지 확인");
        // sort(res.begin(), res.end());
        return res;
    }
}

5.13 Tonelli-Shanks
inline ll multiply(ll a, ll b, ll mod) { return __int128(a) *
b % mod; }
11 binpow(ll a, ll n, ll mod) { //a ^ n % mod
    ll res = 1;
    while (n) {

```

```

        if (n & 1) res = multiply(res, a, mod);
        a = multiply(a, a, mod);
        n >>= 1;
    }
    return res;
}

// p=4k+3이고 n이 이차잉여라면 이산제곱근은 n^((p+1)/4)
11 tonelliShanks(ll n, ll p) { // O(log^2(p)) // x^2 == n
(mod p) // 해가 없다면 -1 리턴
    assert (n >= 0 && n < p);
    if (!n) return 0;
    if (p == 2) return n;
    if (binpow(n, (p - 1) / 2, p) != 1) return -1;
    ll Q = p - 1, S = 0;
    while (~Q & 1) {
        Q >>= 1;
        ++S;
    }
    ll z = 2;
    while (binpow(z, (p - 1) / 2, p) != p - 1) ++z;
    ll M = S;
    ll c = binpow(z, Q, p);
    ll t = binpow(n, Q, p);
    ll R = binpow(n, (Q + 1) / 2, p);
    while (t != 1) {
        ll i = [&]() {
            ll tmp = t;
            for (int i = 1; i < M; i++) {
                tmp = multiply(tmp, tmp, p);
                if (tmp == 1) return i;
            }
        }();
        ll b = binpow(c, binpow(2, M - i - 1, p - 1), p); // c
^ (2 ^ (M - i - 1)), phi(p) = p - 1
        M = i;
        c = multiply(b, b, p);
        t = multiply(t, c, p);
        R = multiply(R, b, p);
    }
    return R; // (p - R) % p도 해가 되긴 함
}

5.14 Cornacchia
11 isqrt(ll n) {
    assert(n >= 0);
    if (n == 0) return 0LL;
    ll x = sqrtl(n);
    while ((x + 1) <= n / (x + 1)) ++x;
    while (x > n / x) --x;
    return x;
}

p11 cornacchia(ll m, ll d) { // x^2 + d y^2 = m // m은 소수,
d>0 // 해 없으면 {-1, -1} // O(log^2 m)
    assert(m >= 2 && d > 0);
    ll r0 = tonelliShanks((m - d % m) % m, m);
    if (r0 == -1) return {-1, -1};
    ll prv = m, cur = r0;
    while ((__int128)cur * cur >= m) {
        ll nxt = prv % cur;
        prv = cur;
        cur = nxt;
    }
    ll x = cur;
    ll rem = m - x * x;
    if (rem < 0 || rem % d != 0) return {-1, -1};
    ll y2 = rem / d;
    ll y = isqrt(y2);
    if ((__int128)y * y != y2) return {-1, -1};
    return {x, y};
}

5.15 Floorsum
11 f(int p, int q, int n) { // p/q + 2p/q + ... + np/q //
O(log(min(p, q)))
    assert(q != 0);
    if (p == 0) return 0;
    if (p >= q) return ll(p / q) * n * (n + 1) / 2 + f(p % q, q,
n);
    return ll(n) * p / q * n + n / q - f(q, p, ll(p) * n / q);
}

11 floorModSum(int p, int q, int n) { // p % q + 2p % q + ...
+ np % q // O(log(min(p, q)))
    int g = __gcd(p, q);

```

```
int pg = p / g, qg = q / g;
return ll(p) * n * (n + 1) / 2 - q * f(pg, qg, n);
}
```

5.16 Xudyh's Sieve

```
template <typename T, T MOD>
class Xudyh {
    vector<T> pfsF; // precomputation
    map<ll, T> mp; // memoization
    T positiveMod(T x) {
        return (x % MOD + MOD) % MOD;
    }
    T pfs_fg(ll x) { // (f * g)(x) =
        // x %= MOD;
        // return positiveMod();
    };
    T pfs_g(ll x) { // g(x) =
        // x %= MOD;
        // return positiveMod();
    };
    T pfs_f(ll x) { // f(x) =
        if (x < pfsF.size()) return pfsF[x]; // precomputation
        if (mp.count(x)) return mp[x];
        auto &res = mp[x];
        res = pfs_fg(x); // S_f(N) = ( S_f * g(N) - sum {d =
        2~N} g(d)S_f(N/d) ) / g(1)
        for (ll i = 2, j; i <= x; i = j + 1) {
            j = x / (x / i);
            res -= pfs_f(x / i) * (pfs_g(j) - pfs_g(i - 1) +
            MOD) % MOD;
            res %= MOD;
        }
        res = (res + MOD) % MOD;
        res = res * modInv(g(1), MOD) % MOD; // res /= g(1)
        return res;
    }
public:
    Xudyh(ll maxN) {
        int preComputeSize = pow(2.0L * maxN, 2.0L / 3);
        pfsF = getMulFunc(preComputeSize);
        for (int i = 1; i < pfsF.size(); i++) pfsF[i] =
        (pfsF[i] + pfsF[i - 1]) % MOD;
    }
    T getSum(ll i, ll j) { // O(N^{2/3}) // precomputation 안
    할 경우 O(N^{3/4}) // precomputation K까지했을 때 O(K +
    \frac{2N}{\sqrt{K}})
        return positiveMod(pfs_f(j) - pfs_f(i - 1));
    }
};
// g(1)S_f(N) = S_{f*g}(N) - sum_{d=2}^N g(d) S_f(N/d)
// $S_f(N) = (S_{f*g}(N) - sum_{d=2}^N g(d) S_f(N/d)) /
g(1)$
```

5.17 Lagrange Interpolation

```
ll pm(ll n, ll mod) {
    return (n % mod + mod) % mod;
}
// f(0), ..., f(n) 알고 있을 때 n차 다항식 f(x)의 x좌표에서
함수값 계산
ll lagrangeInterpolation(vector<ll> y, ll x, ll mod) { // O(N
\log{MOD})
    assert(!y.empty());
    for (auto &e : y) e = pm(e, mod);
    x = pm(x, mod);
    int n = y.size() - 1;
    if (x <= n) return pm(y[x], mod);
    ll k = 1; // PI_{j!=i} (x - x_j) / (x_i - x_j) 저장
    for (int j = 1; j < y.size(); j++) { // i = 0 // j != i
        k = k * pm(x - j, mod) % mod; // * (x - x_j)
        k = k * modInv(pm(-j, mod), mod) % mod; // / (x_i -
        x_j)
    }
    ll res = 0;
    for (int i = 0; i <= n; i++) {
        res = (res + y[i] * k) % mod;
        if (i < n) {
            k = k * pm(x - i, mod) % mod * modInv(pm(x - (i +
            1), mod), mod) % mod;
            k = k * pm(i - n, mod) % mod * modInv(pm(i + 1,
            mod), mod) % mod;
        }
    }
    return res;
}
```

```
}
// (x_0, y_0), ..., (x_n, y_n)을 지나는 n차함수:
// y=sum_{i=0}^n [ y_i prod_{j \neq i}(x-x_j) / prod_{j \neq
i}(x_i-x_j) ]
```

```
ll powerSum(ll n, int k, ll mod) { // O(K log{MOD}) + O(K
logK) // (1^k + 2^k + ... + n^k) mod M
    vector<ll> y(k + 2, 0);
    for (int x = 1; x <= k + 1; x++) y[x] = (y[x - 1] +
    power(x, k, mod)) % mod;
    return n < y.size() ? y[n] : lagrangeInterpolation(y, n,
    mod);
}
```

5.18 Moebius Inversion

$\tau(n)$: 양의 약수의 개수

$\sigma(n)$: 양의 약수의 합

$\phi(n)$: n 이하의 자연수 중 n 과 서로소인 수의 개수 ($\phi(1) = 1$)

$\mu(n)$: 뫼비우스 함수

$\varepsilon(n)$: $[n = 1]$ (Iverson bracket)

$1(n) = 1, \text{Id}(n) = n$

Dirichlet Convolution

$$\tau = 1 * 1, \quad \sigma = \text{Id} * 1, \quad \text{Id} = \phi * 1, \quad 1 = \varepsilon * 1, \quad \sigma = \phi * \tau$$

Moebius Inversion

$$g(n) = \sum_{d|n} f(d), \quad \forall n \in \mathbb{N} \iff f(n) = \sum_{d|n} g(d) \mu\left(\frac{n}{d}\right), \quad \forall n \in \mathbb{N}$$

즉,

$$g = f * 1 \iff f = g * \mu$$

$$1 = \tau * \mu, \quad \text{Id} = \sigma * \mu, \quad \phi = \text{Id} * \mu, \quad \varepsilon = 1 * \mu$$

$$\varepsilon = (\text{Id} \mu) * \text{Id}$$

$$\text{Id}^{k+1} = (\text{Id}^k \phi) * \text{Id}^k$$

GCD Sums

$$\sum_{k=1}^n \sum_{d=1}^{\lfloor n/k \rfloor} = \sum_{l=1}^n \sum_{d|l} \left(l = kd, d = \frac{l}{k} \right)$$

1.

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

1'.

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = g] = \sum_{d=1}^{\lfloor n/g \rfloor} \mu(d) \left\lfloor \frac{n}{gd} \right\rfloor^2$$

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = 2 \sum_{i=1}^n \phi(i) - 1$$

2.

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) &= \sum_{k=1}^n k \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = k] \\ &= \sum_{k=1}^n k \sum_{i=1}^{\lfloor n/k \rfloor} \sum_{j=1}^{\lfloor n/k \rfloor} [\gcd(i, j) = 1] \\ &= \sum_{k=1}^n k \sum_{d=1}^{\lfloor n/k \rfloor} \mu(d) \left\lfloor \frac{\lfloor n/k \rfloor}{d} \right\rfloor^2 \\ &= \sum_{k=1}^n k \sum_{d=1}^{\lfloor n/k \rfloor} \mu(d) \left\lfloor \frac{n}{kd} \right\rfloor^2 \\ &= \sum_{l=1}^n \left\lfloor \frac{n}{l} \right\rfloor^2 \sum_{k|l} k \mu\left(\frac{l}{k}\right) \\ &= \sum_{l=1}^n \left\lfloor \frac{n}{l} \right\rfloor^2 \phi(l) \end{aligned}$$

2'.

$$\sum_{i=1}^n \sum_{j=1}^m \gcd(i, j) = \sum_{l=1}^{\min(n, m)} \left\lfloor \frac{n}{l} \right\rfloor \left\lfloor \frac{m}{l} \right\rfloor \phi(l)$$

2''.

$$\sum_{i=1}^n \sum_{j=1}^m i \gcd(i, j) = \sum_{l=1}^{\min(n, m)} \left\lfloor \frac{m}{l} \right\rfloor \frac{\lfloor n/l \rfloor (\lfloor n/l \rfloor + 1)}{2} l \phi(l)$$

2'''.

$$\sum_{i=1}^n \sum_{j=1}^m ij \gcd(i, j) = \sum_{l=1}^{\min(n, m)} \frac{\lfloor m/l \rfloor (\lfloor m/l \rfloor + 1)}{2} \frac{\lfloor n/l \rfloor (\lfloor n/l \rfloor + 1)}{2} l^2 \phi(l)$$

5.19 Math Theory

$$\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$$

$$\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$$

$$\sum_{x=0}^n \binom{x+m}{m} = \binom{n+m+1}{m+1}$$

$$\sum_{x=0}^n x \binom{x+m}{m} = \sum_{x=0}^n x \binom{x+m}{x} = \sum_{x=0}^n (m+1) \binom{x+m}{m+1} = (m+1) \binom{n+m+1}{m+2}$$

$$\sum_{x=0}^n x^2 \binom{x+m}{m} = \sum_{x=1}^n (m+1)x \binom{x+m}{m+1} = (m+1) \sum_{x=0}^{n-1} (x+1) \binom{x+m+1}{m+1}$$

$$1) \binom{x+m+1}{m+1} = (m+1)(m+2) \binom{x+m+1}{m+3} + (m+1) \binom{x+m+1}{m+2}$$

$$\binom{n}{r} \binom{r}{k} = \binom{n}{k} \binom{n-k}{r-k} \quad (0 \leq k \leq r \leq n)$$

Canonical Coin System

$c_i = \frac{r^i - 1}{r - 1}$ 은 $\forall r \geq 2$ 에 대해 canonical coin system임. ex) {1, 3, 7, 15}

Jacobsthal's function

n	P_n	Primorial ($P_n\#$)	Jacobsthal $j(P_n\#)$
1	2	2	2
2	3	6	4
3	5	30	6
4	7	210	10
5	11	2,310	14
6	13	30,030	22
7	17	510,510	26
8	19	9,699,690	34
9	23	223,092,870	40
10	29	6,469,693,230	46

$$\phi(n) \approx \frac{6}{\pi^2} n \approx 0.6079n$$

Maximal Prime Gaps

Upper Bound (N)	Max Gap g_n	Starting Prime p_n
10^3	20	887
10^6	114	492,113
10^9	282	473,266,931
10^{12}	906	999,999,999,763
10^{15}	1096	908,460,772,044,959
10^{18}	1,476	1,425,172,824,437,699,411
2^{64}	1,550	18,361,375,334,787,046,697

Bertrand's Postulate $\forall n > 1, \exists p$ s.t. $n < p < 2n$

moebius

n 이 square-free 수의 개수 $\sum_{i=1}^n \mu[i] \left\lfloor \frac{n}{i^2} \right\rfloor$

Frobenius' coin problem

서로소인 두 자연수 a, b 에 대해 $an + bm$ 꼴로 나타낼 수 없는 최대값은 $ab - a - b$

즉, $ab - a - b + 1 = (a-1)(b-1)$ 이상의 자연수는 전부 $an + bm$ 꼴로 표현 가능

$\gcd(a, b) = g$ 라면 $(a/g - 1)(b/g - 1)g$ 이상의 g 의 배수는 전부 표현 가능

Cycle Count Generating Functions

순열 $\sigma \in S_n$ 의 cycle 개수를 $c(\sigma)$, 교란순열의 집합을 $\mathcal{D}_n$ 이라 하면

$$\sum_{\sigma \in S_n} x^{c(\sigma)} = x(x+1)(x+2) \cdots (x+n-1),$$

$$\sum_{\sigma \in \mathcal{D}_n} x^{c(\sigma)} = \sum_{q=0}^n (-1)^q \binom{n}{q} x^q \prod_{i=0}^{n-q-1} (x+i).$$

$$\sum_{\sigma \in S_n} (-1)^{c(\sigma)} = \begin{cases} 1 & n=0, \\ -1 & n=1, \\ 0 & n \geq 2, \end{cases} \quad \sum_{\sigma \in \mathcal{D}_n} (-1)^{c(\sigma)} = 1 - n.$$

Non-adjacent Selection

n 개 중 r 개를 서로 인접하지 않게 고르는 경우의 수 $\binom{n-r+1}{r}$

Pick's Theorem 격자 다각형 넓이 A , 내부 격자점 개수 I , 경계 격자점 개수 B

$$A = I + \frac{B}{2} - 1.$$

$$B = \sum_{\text{edges}} \gcd(|x_2 - x_1|, |y_2 - y_1|).$$

6 String

6.1 Rolling Hash

```
template<ll MOD=1'000'000'007>
struct Hashing {
    vector<ll> h;
    static const ll X;
    static vector<ll> x;
    template <typename T> // T = string or vector<
    Hashing(const T &st) : h(st.size() + 1) {
        while (x.size() <= st.size()) x.push_back(x.back() * X
        % MOD);
        for (int i = 1; i < h.size(); i++) h[i] = (h[i - 1] *
        X + st[i - 1]) % MOD;
    }
    ll get(int s, int e) const { // 0-based, st[s:e]
        ll res = (h[e] - h[s] * x[e - s]) % MOD; // h는
        1-based
        return res >= 0 ? res : res + MOD;
    }
};

template<ll MOD> const ll Hashing<MOD>::X = sqrt(MOD) -
(__TIME__[6] & 15) * 10 - (__TIME__[7] & 15);
template<ll MOD> vector<ll> Hashing<MOD>::x = {1};
// 2D
template<ll MOD=1'000'000'007>
struct Hashing2D {
    vector<vector<ll>> h;
    static const ll X, Y;
    static vector<ll> x, y;
    template <typename T> // T = string or vector<
    Hashing2D(const vector<T> &v) {
        int n = v.size();
        int m = v[0].size();
        while (x.size() <= n) x.push_back(x.back() * X % MOD);
        while (y.size() <= m) y.push_back(y.back() * Y % MOD);
        h.assign(n + 1, vector<ll>(m + 1));
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                h[i][j] = v[i - 1][j - 1]
                + h[i - 1][j] * X % MOD
                + h[i][j - 1] * Y % MOD
                - h[i - 1][j - 1] * X % MOD * Y % MOD;
                h[i][j] %= MOD;
                if (h[i][j] < 0) h[i][j] += MOD;
            }
        }
    }
    ll get(int i1, int i2, int j1, int j2) const { // 0-based,
    v[i1:i2][j1:j2]
        ll res = h[i2][j2]
        - h[i1][j2] * x[i2 - i1] % MOD
        - h[i2][j1] * y[j2 - j1] % MOD
```

```

        + h[i1][j1] * x[i2 - i1] % MOD * y[j2 - j1] %
        MOD;
    res %= MOD;
    return res >= 0 ? res : res + MOD;
}
};
template<ll MOD> const ll Hashing2D<MOD>::X = sqrt(MOD) -
(__TIME__[6] & 15) * 10 - (__TIME__[7] & 15);
template<ll MOD> const ll Hashing2D<MOD>::Y = sqrt(MOD) +
(__TIME__[6] & 15) * 10 + (__TIME__[7] & 15);
template<ll MOD> vector<ll> Hashing2D<MOD>::x = {1};
template<ll MOD> vector<ll> Hashing2D<MOD>::y = {1};

```

6.2 KMP

```

template <typename T>
vector<int> getPi(const T &st) { // O(N)
    vector<int> pi(st.size());
    for (int i = 1, j = 0; i < st.size(); i++) {
        while (j > 0 && st[i] != st[j]) j = pi[j - 1];
        if (st[i] == st[j]) pi[i] = ++j;
    }
    return pi;
}
template <typename T>
vector<int> kmp(const T &st, const T &pattern) { // O(N+M)
    int n = st.size(), m = pattern.size();
    auto pi = getPi(pattern);
    vector<int> res;
    for (int i = 0, j = 0; i < n; i++) {
        while (j > 0 && st[i] != pattern[j]) j = pi[j - 1];
        if (st[i] == pattern[j]) {
            if (j == m - 1) {
                res.push_back(i - m + 1);
                j = pi[j];
            }
            else ++j;
        }
    }
    return res;
}
// 최소주기: n - pi.back() // ex) st = "abaab", pi.back() = 2
// cyclic string에서 검색: kmp(st + st, pattern)

```

6.3 Aho-Corasick

```

template <int ALPHA = 26, char FIRST = 'a'>
struct Aho {
    struct Node {
        array<int, ALPHA> mp, go;
        int fail = 0;
        bool end = false;
        Node() {
            mp.fill(-1);
            go.fill(-1);
        }
    };
    vector<Node> tree = {Node()};
    int charToIdx(char ch) const { return ch - FIRST; }
    void insert(const string &st) {
        int cur = 0;
        for (auto ch : st) {
            int i = charToIdx(ch);
            if (!tree[cur].mp[i]) {
                tree[cur].mp[i] = tree.size();
                tree.emplace_back();
            }
            cur = tree[cur].mp[i];
        }
        tree[cur].end = true;
    }
    void build() { // build O(ALPHA * sum(m_i))
        queue<int> q;
        for (int i = 0; i < ALPHA; i++) {
            int nxt = tree[0].mp[i];
            if (!nxt) tree[0].go[i] = 0;
            else {
                tree[0].go[i] = nxt;
                q.push(nxt);
            }
        }
        while (!q.empty()) {
            int cur = q.front();
            q.pop();
            tree[cur].end |= tree[tree[cur].fail].end;
        }
    }
};

```

```

for (int i = 0; i < ALPHA; i++) {
    int nxt = tree[cur].mp[i];
    if (!nxt) tree[cur].go[i] =
    tree[tree[cur].fail].go[i];
    else {
        tree[cur].go[i] = nxt;
        tree[nxt].fail =
        tree[tree[cur].fail].go[i];
        q.push(nxt);
    }
}
}
bool findSubstring(const string &st) const { // O(N)
    int cur = 0;
    for (auto ch : st) {
        cur = tree[cur].go[charToIdx(ch)];
        if (tree[cur].end) return true;
    }
    return false;
}
};

```

6.4 Z

```

template <typename Container>
vector<int> get_z(const Container &st) { // O(N)
    vector<int> z(st.size());
    for (int i = 1, p = 0; i < st.size(); i++) {
        if (i < p + z[p]) z[i] = min(p + z[p] - i, z[i - p]);
        while (i + z[i] < st.size() && st[z[i]] == st[i +
        z[i]]) ++z[i];
        if (p + z[p] < i + z[i]) p = i;
    }
    z[0] = st.size();
    return z;
} // z[i] = lcp(st, st[i:])

```

6.5 Manacher

```

template <typename Container>
vector<int> manacher(const Container &orig, typename
Container::value_type dummy) { // O(N)
    Container st(orig.size() << 1 | 1, dummy);
    for (int i = 0; i < orig.size(); i++) st[2 * i + 1] =
    orig[i];
    vector<int> r(st.size());
    for (int i = 1, p = 0; i < st.size(); i++) {
        if (i < p + r[p]) r[i] = min(p + r[p] - i, r[2 * p -
        i]);
        while (i - r[i] - 1 >= 0 && i + r[i] + 1 < st.size()
        && st[i - r[i] - 1] == st[i + r[i] + 1]) ++r[i];
        if (p + r[p] < i + r[i]) p = i;
    }
    return r;
} // 중심st[i]인 가장 긴 팰린드롬 길이 : r[2i+1],
중심st[i:i+1] : r[2i+2]

```

6.6 Suffix Array

```

template <typename Container>
auto get_sa(const Container &st) { // O(N log N)
    int n = st.size();
    if (n == 1) return pair{vector{0}, vector{0}};
    assert(n > 0);
    int m = *max_element(st.begin(), st.end());
    vector<int> sa(n), x(n + 1), y(n + 1), cnt(max(n, m) + 1);
    for (int i = 0; i < n; i++) assert(st[i] > 0);
    for (int i = 0; i < n; i++) ++cnt[x[i] = st[i]];
    for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--) sa[--cnt[x[i]]] = i;
    for (int len = 1, p = 1; p < n; len <= 1, m = p) {
        int idx = 0;
        for (int i = n - len; i < n; i++) y[idx++] = i;
        for (int i = 0; i < n; i++) if (sa[i] >= len) y[idx++]
        = sa[i] - len;
        fill(cnt.begin(), cnt.begin() + m + 1, 0);
        for (int i = 0; i < n; i++) ++cnt[x[y[i]]];
        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) sa[--cnt[x[y[i]]]] =
        y[i];
        swap(x, y);
        x[sa[0]] = p = 1;
        for (int i = 1; i < n; i++) {
            int a = sa[i - 1], b = sa[i];
            x[b] = y[a] == y[b] && y[a + len] == y[b + len] ?
            p : ++p;
        }
    }
}

```

```

    }
    x.resize(n);
    for (auto &e : x) --e;
    return pair{sa, x}; // rank=x
}
template <typename Container>
vector<int> get_lcp(const Container &st, const vector<int>
&sa, const vector<int> &rank) { // O(N)
    int n = st.size();
    assert(n >= 1);
    vector<int> lcp(n - 1);
    for (int i = 0, h = 0; i < n; ++i, h -= !!h) if (rank[i])
    {
        for (int j = sa[rank[i] - 1]; j + h < n && i + h < n
&& st[j + h] == st[i + h];) ++h;
        lcp[rank[i] - 1] = h;
    }
    return lcp;
} // lcp[i]: sa[i], sa[i + 1]의 가장 공통 접두사 // lcp.size()
= n-1
// auto [sa, rank] = get_sa(st);
// auto lcp = get_lcp(st, sa, rank);
// RMQ<int, min> rmq(lcp);
auto lce = [&](int i, int j) { // lcp(st[i:], st[j:])
    if (i == j) return n - i;
    int idx1 = rank[i];
    int idx2 = rank[j];
    if (idx1 > idx2) swap(idx1, idx2);
    return rmq.query(idx1, idx2 - 1); // i!=j라서 idx1 <= idx2
- 1 성립
};
auto cmp = [&](int i, int j, int i2, int j2) { // st[i:j] <
st[i2:j2]
    int len = lce(i, i2);
    if (len >= min(j - i + 1, j2 - i2 + 1)) return j - i < j2
- i2;
    return rank[i] < rank[i2];
};

```

6.7 String Theory

Infinite Order

비어 있지 않은 문자열 a, b 에 대해 사전순으로 $a^\infty < b^\infty \iff ab < ba$

7 Misc

7.1 Gray Code

```

void grayCode(int n) {
    for (int i = 0; i < (1 << n); ++i) cout << (i ^ (i >> 1))
<< "\n";
}

```

7.2 Majority Vote

```

template <typename T>
pair<bool, T> boyerMoore(const vector<T> &v) { // O(N)
    assert(!v.empty());
    int cnt = 0;
    T candi;
    for (auto e : v) {
        if (!cnt) candi = e;
        if (e == candi) ++cnt;
        else --cnt;
    }
    if (cnt) {
        cnt = 0;
        for (auto e : v) cnt += (e == candi);
    }
    return {cnt > v.size() / 2, candi};
}

```

7.3 Ternary Search

```

auto ternarySearch = [&](ll l, ll r) { // min
    ll lo = l, hi = r;
    while (hi - lo >= 3) {
        ll p = (lo * 2 + hi) / 3, q = (lo + hi * 2) / 3;
        if (f(p) > f(q)) lo = p; // max0이면 if (f(p) < f(q))
        else hi = q;
    }
    auto res = f(lo);
    for (ll i = lo + 1; i <= hi; ++i) res = min(res, f(i));
    return res;
};

```

7.4 RMQ

```

template <typename T, const T& (*op)(const T&, const T&>
struct RMQ {
    vector<vector<T>> ac; // ac[i][j] = op(v[j:j+2^i))
    RMQ(const vector<T> &v) : ac(1, v) { // O(N logN)
        for (int i = 1, len = 1; len * 2 <= v.size(); ++i, len
*= 2) {
            ac.emplace_back(v.size() - len * 2 + 1);
            for (int j = 0; j < ac[i].size(); j++) ac[i][j] =
op(ac[i - 1][j], ac[i - 1][j + len]);
        }
    }
    T query(int a, int b) const { // 0-based // op[a:b]의 op()
누적값 // O(1)
        assert(0 <= a && a <= b && b < ac[0].size());
        int i = __lg(b - a + 1);
        return op(ac[i][a], ac[i][b - (1 << i) + 1]);
    }
};
// const RMQ<int, min> rmq(v);

```

7.5 Deque Trick

```

vector<int> dequeTrickMin(int len, const vector<int> &v) { //
res[i]는 v[max(0, i - len + 1)]~v[i]의 최솟값 // 즉, 정확히
길이가 len인 구간의 최솟값은 res[len-1:n]에 저장됨
    vector<int> res(v.size());
    deque<int> dq;
    for (int i = 0; i < v.size(); i++) {
        while (!dq.empty() && v[dq.back()] > v[i])
            dq.pop_back(); // min
        // while (!dq.empty() && v[dq.back()] < v[i])
            dq.pop_back(); // max
        dq.push_back(i);
        if (i - dq.front() + 1 > len) dq.pop_front();
        res[i] = v[dq.front()];
    }
    return res;
}
// cmp 복잡할때
deque<int> dq;
auto cmp = [&](int front, int back) { // front에 구하려는 값이
들어가야 함
    // ex) min0이면 return v[front] <= v[back];
};
for (int i = 0; i < v.size(); i++) {
    while (!dq.empty() && !cmp(dq.back(), i)) dq.pop_back();
    dq.push_back(i);
    if (i - dq.front() + 1 > len) dq.pop_front();
}

```

7.6 Random

```

mt19937 rng;
shuffle(v.begin(), v.end(), rng); // O(N)
uniform_int_distribution<int> dist(a, b); // a<=x<=b
uniform_real_distribution<double> dist(a, b); // a<=x<b
x = dist(rng);

```

7.7 Time

```

clock_t sttime = clock();
while(double(clock() - sttime) / CLOCKS_PER_SEC < 0.95) {}

```

7.8 Debug

```

g++ -std=c++17 -O0 -g -fsanitize=undefined
-fno-omit-frame-pointer main.cpp -o main
# -fsanitize=address

```